Árboles y árboles binarios

La recursión desde la definición, y los tres recorridos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Describir un árbol concreto con su vocabulario: raíz, padre, hijo, hermano, hoja, grado, profundidad, nivel y altura (OA2).
- Escribir una función recursiva sobre un árbol binario a partir de su definición, con un caso base y un caso recursivo, y seguir a mano su pila de llamadas (OA2, OA3).
- Recorrer un árbol binario en preorden, inorden y posorden, primero con recursión y después con una pila explícita (OA3).

Objetivos de aprendizaje II

Al finalizar esta sesión, el estudiante podrá también

- Dar el costo en tiempo y en espacio de cada función sobre el árbol, y explicar por qué un árbol degenerado desborda la pila de llamadas (OA1).
- Resolver problemas de juez que piden recorrer, medir o reconstruir un árbol binario (OA5).

Contenido I

1. Repaso: la tabla hash y el `map`
2. Datos que se contienen unos a otros
3. El TAD árbol
4. El árbol binario
5. La recursión, desde el árbol
6. Recorridos: preorden, inorden y posorden
7. Recorridos con pila explícita
8. Liberar el árbol
9. Ejercicios en parejas
10. Para el juez
11. Lo que cuesta

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

Lo que quedó de la tabla hash y el `map` I

	Buscar, insertar	Recorrer en orden de clave
Tabla hash	$\Theta(1+\alpha)$ en promedio	no: hay que ordenar al final
`map`	$O(\log n)$	sí, en $\Theta(n)$

La tabla hash gana en velocidad promedio y pierde el orden. El `map` paga un $\log n$ y a cambio entrega las claves de menor a mayor sin ordenarlas al final.

Pregunta

¿Qué estructura guarda el `map` por dentro para recorrer sus claves en orden y además buscar en $O(\log n)$?

La respuesta tiene varias piezas. La primera es el árbol binario.

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

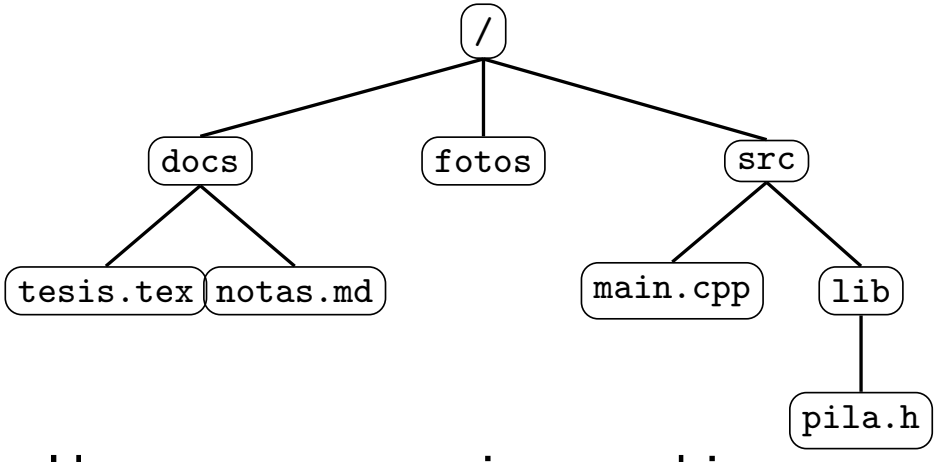
8 Liberar el árbol

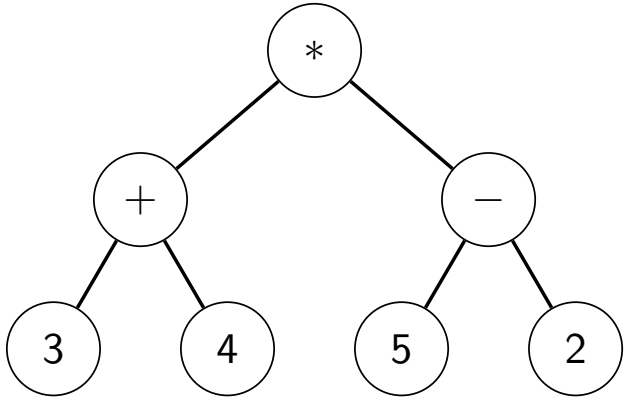
9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

Carpetas y expresiones I





Una carpeta contiene archivos y otras carpetas. En $(3 + 4) * (5 - 2)$ la multiplicación contiene dos operaciones, y cada una contiene dos números.

Carpetas y expresiones II

Pregunta

Se guardan los nueve nombres en una lista enlazada, en el orden en que aparecen al recorrer el disco:

```
/ docs tesis.tex notas.md fotos src main.cpp lib
                                    pila.h
```

¿Qué hay dentro de `src`? ¿En qué carpeta está `pila.h`? ¿Sirve la lista?

Carpetas y expresiones III

Respuesta

La lista guarda un orden, no quién contiene a quién: con ella no se sabe si `main.cpp` está en `src` o en la raíz. Si cada elemento guardara además el nombre de su carpeta, preguntar qué hay en `src` obligaría a recorrer la lista completa, $\Theta(n)$ por pregunta.

Lo que se quiere es que cada carpeta apunte a lo que contiene: desde `src` se baja a `main.cpp` y a `lib` sin mirar lo demás. Con la expresión pasa igual: en la cadena hay que buscar los paréntesis para saber qué se opera primero, y en el dibujo cada operador queda encima de sus dos operandos. Lo que está más abajo se calcula antes.

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

Árbol con raíz I

Árbol con raíz (CLRS, §B.5.2; Thareja, §9.1.1)

- Un nodo se distingue como *raíz*. Todo nodo distinto de la raíz tiene un único *padre*; los nodos que lo tienen como padre son sus *hijos*, y dos nodos con el mismo padre son *hermanos*.
- Una *hoja* no tiene hijos. Un *nodo interno* tiene al menos uno. El *grado* de un nodo es su número de hijos.
- El *subárbol* de un nodo x es x con todos los nodos que cuelgan de él.

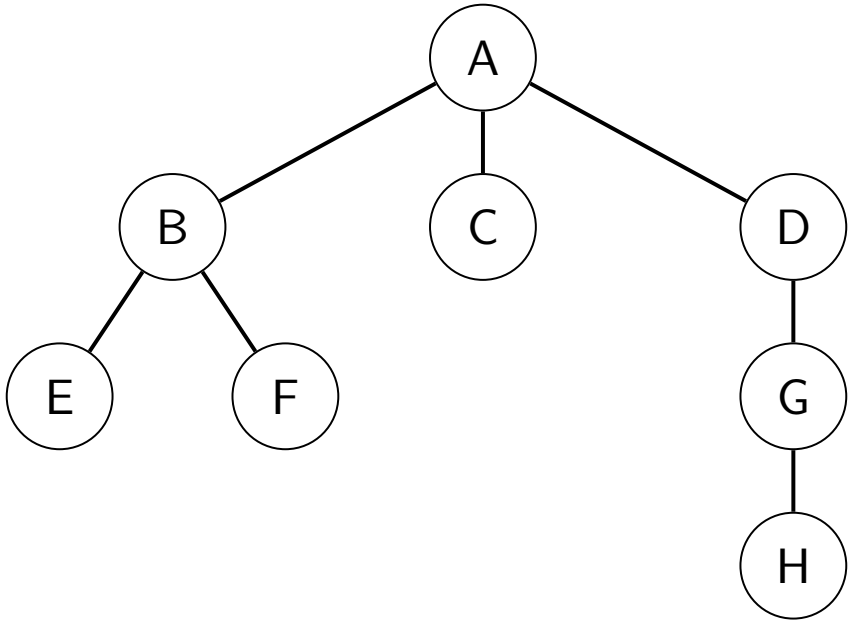
Árbol con raíz II

Medidas (CLRS, §B.5.2)

- *Profundidad* de x: número de aristas del camino de la raíz a x. La raíz tiene profundidad 0.
- *Nivel* k: los nodos de profundidad k.
- *Altura* de x: número de aristas del camino más largo que baja de x hasta una hoja. Una hoja tiene altura 0, y la altura del árbol es la de su raíz.

La profundidad se cuenta desde arriba y la altura desde abajo.

Un árbol concreto I



Pregunta

¿Quién es el padre de G y quiénes son los hermanos de C? ¿Cuál es el grado de A? ¿Qué profundidad tiene H y qué altura tiene D? ¿Qué nodos forman el nivel 2? ¿Cuál es la altura del árbol?

Un árbol concreto II

Nodo	Padre	Hijos	Grado	Profundidad	Altura	Tipo
A	ninguno	B, C, D	3	0	3	raíz
B	A	E, F	2	1	1	interno
C	A	ninguno	0	1	0	hoja
D	A	G	1	1	2	interno
E	B	ninguno	0	2	0	hoja
F	B	ninguno	0	2	0	hoja
G	D	H	1	2	1	interno
H	G	ninguno	0	3	0	hoja

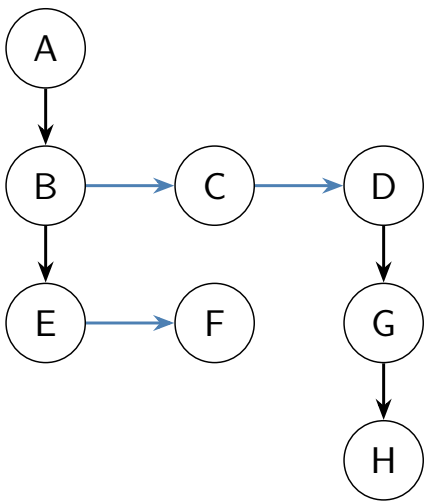
Los hermanos de C son B y D. El nivel 2 es $\{E, F, G\}$ y la altura del árbol es 3, la de A, por el camino A, D, G, H.

Aristas, no nodos

D tiene profundidad 1 y altura 2: no son la misma medida. Las dos cuentan aristas. Algunos enunciados cuentan nodos, y entonces una hoja tiene altura 1; antes de programar se lee cuál usa.

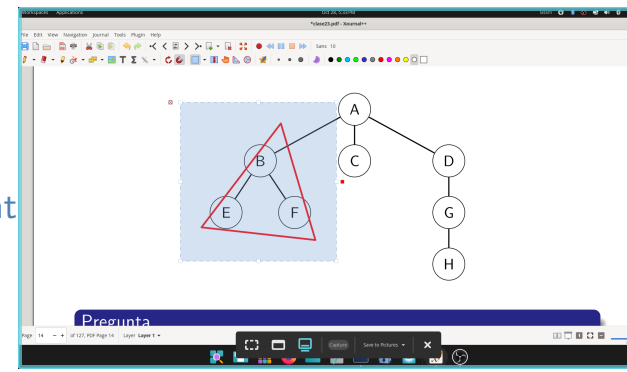
Hijo izquierdo, hermano derecho I

Un nodo puede tener cualquier número de hijos, y un arreglo de hijos de tamaño fijo se queda corto o desperdicia casillas. CLRS (§10.3) guarda cualquier árbol con raíz usando dos punteros por nodo: uno a su primer hijo y otro a su hermano siguiente.



↓ primer hijo

→ hermano siguient



Con dos punteros por nodo, cualquier árbol con raíz se guarda como un árbol binario. Listar los hijos de x cuesta $\Theta(\text{grado de } x)$: se baja al primero y se avanza por los hermanos. Desde aquí la sesión trabaja con árboles binarios.

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

Definición I

Árbol binario (CLRS, §B.5.3; Thareja, §9.2.3)

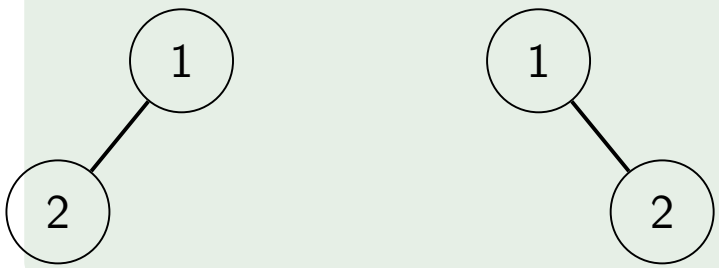
Un árbol binario es una de dos cosas:

- el *árbol vacío*, que no tiene nodos, o
- un nodo raíz junto con dos árboles binarios disjuntos: el *subárbol izquierdo* y el *subárbol derecho*.

La definición usa la palabra que está definiendo: un árbol binario se arma con árboles binarios más pequeños, y el vacío es donde se acaba.

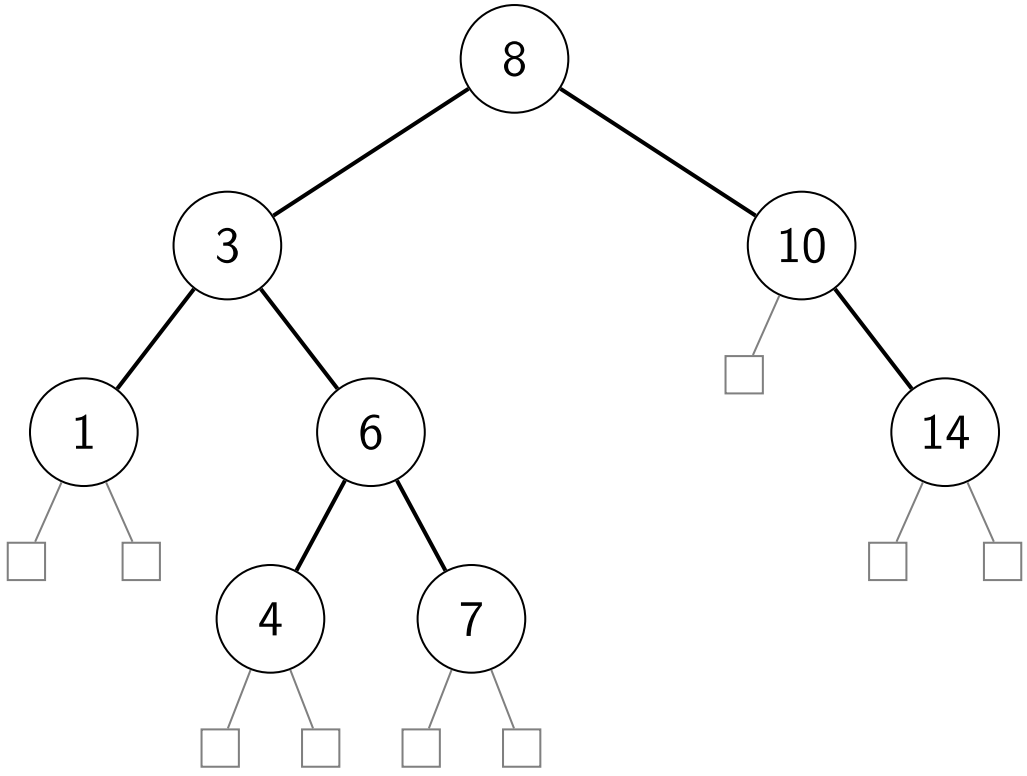
Definición II

Izquierdo y derecho cuentan



Como árboles con raíz son el mismo.
Como árboles binarios son distintos:
en el primero el 2 es hijo izquierdo,
en el segundo es hijo derecho.

El árbol de ejemplo, con sus vacíos I



Ocho nodos. Cada hijo que falta es un subárbol vacío, dibujado como un cuadro gris: hay nueve. El 10 tiene un solo hijo, el derecho. Este árbol se usa en todas las trazas de la sesión.

El nodo en C++ I

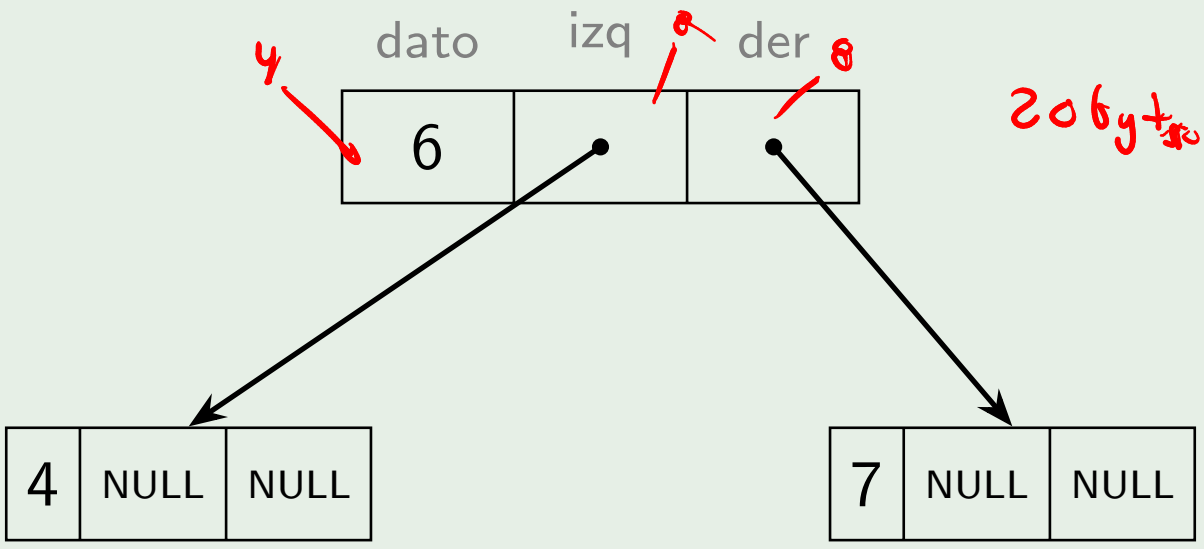
```
struct Nodo {
  int dato;
  Nodo *izq;  // raiz del subarbol izquierdo
  Nodo *der;  // raiz del subarbol derecho
};

Nodo *crearNodo(int dato, Nodo *izq, Nodo *der) {
  Nodo *nuevo = new Nodo;
  nuevo->dato = dato;
  nuevo->izq = izq;
  nuevo->der = der;
  return nuevo;
}
```

Un árbol se maneja con el puntero a su raíz, y el árbol vacío es `NULL`. `crearNodo` cuesta $\Theta(1)$ en tiempo y en espacio: un `new` y tres asignaciones.

El nodo en C++ II

El subárbol de 6 en memoria



La estructura ocupa $\Theta(n)$: un nodo por dato, con el dato y dos punteros.

Armar el árbol a mano I

```
Nodo *seis = crearNodo(6, crearNodo(4, NULL, NULL), crearNodo(7,
↪  NULL, NULL));
Nodo *tres = crearNodo(3, crearNodo(1, NULL, NULL), seis);
Nodo *diez = crearNodo(10, NULL, crearNodo(14, NULL, NULL));
Nodo *raiz = crearNodo(8, tres, diez);
```

Pregunta

¿Por qué el código empieza por el 6 y deja el 8 para el final?

Armar el árbol a mano II

Respuesta

`crearNodo` recibe los dos hijos ya hechos. Para crear el 8 hacen falta el 3 y el 10, y para crear el 3 hace falta el 6: el árbol se arma de abajo hacia arriba. Las hojas se crean con `NULL` en los dos punteros.

Al final del programa, `liberar(raiz)` devuelve los ocho nodos. El orden en que lo hace sale de los recorridos.

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

De la definición a la función I

La definición del árbol binario tiene dos casos, y una función sobre árboles binarios tiene los mismos dos.

Caso base y caso recursivo

- **Caso base**: el árbol vacío. La respuesta se da directamente, sin llamar a nadie.
- **Caso recursivo**: un nodo con dos subárboles. La función se llama sobre cada subárbol, que es más pequeño, y combina lo que devuelven con el dato de la raíz.

Pregunta

¿Cuántos nodos tiene un árbol binario? Responda con los dos casos, en palabras, antes de ver el código.

De la definición a la función II

Respuesta

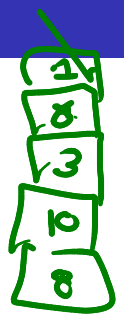
El vacío tiene 0 nodos. Un árbol con raíz tiene la raíz más los nodos de sus dos subárboles:

$$tamano(T) = \begin{cases} 0 & \text{si } T \text{ es vacío} \\ 1 + tamano(I) + tamano(D) & \text{si no} \end{cases}$$

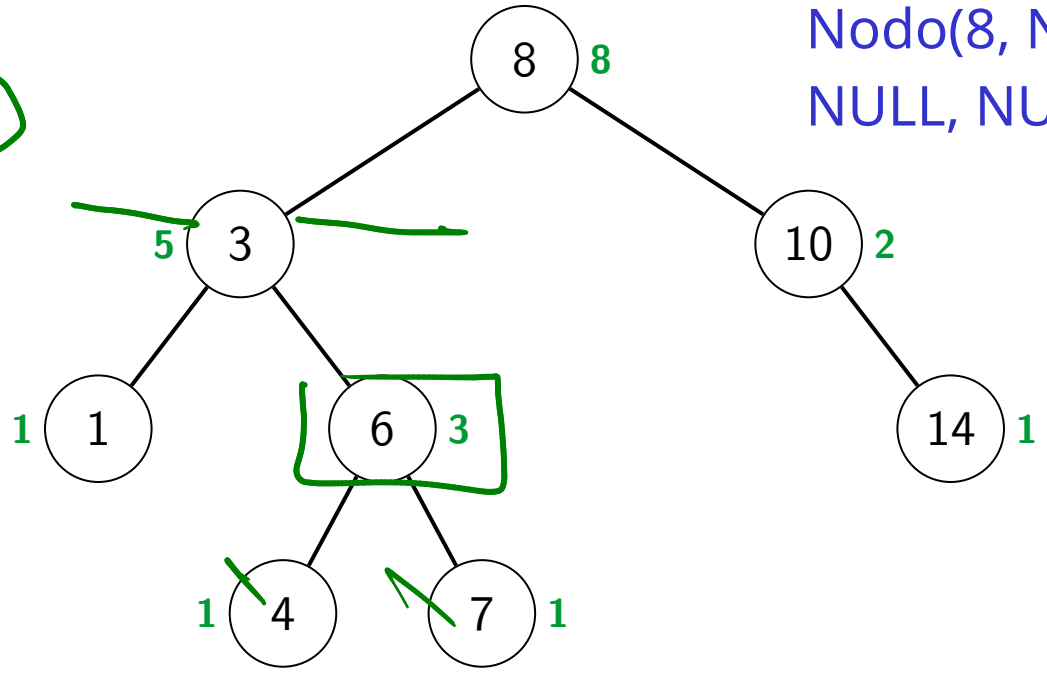
```
// Numero de nodos. El arbol vacio tiene 0.
int tamano(Nodo *r) {
  int res = 0;
  if (r != NULL) {
    res = 1 + tamano(r->izq) + tamano(r->der);
  }
  return res;
}
```

Cada línea de la fórmula es una línea del código. `res = 0` es el caso base: si `r` es `NULL`, el `if` no entra.

Lo que devuelve cada llamada I







Nodo(8, Nodo(3, Nodo(1, NULL, NULL), Nodo(6, ...

En verde, lo que devuelve `tamano` sobre el subárbol de cada nodo; `tamano(raiz)` devuelve 8.

Pregunta

`tamano(3)` recibe 1 de `tamano(1)` y 3 de `tamano(6)`. ¿Qué devuelve?

T(a) = 1 + T(3) + T(10)

T(a) = 1 + 1 + T(1) + T(6) + T(10)

T(a) = 1 + 1 + 1 + T(NULL) + T(NULL)+ T(6) + T(10)

T(a) = 1 + 1 + 1 + 0 + 0 + T(6) + T(10)

T(a) = 1 + 1 + 1 + 0 + 0 + 1 + T(4) + T(7) + T(10)

T(a) = 1 + 1 + 1 + 0 + 0 + 1 + 1 + T(NULL) + T(NULL) + T(7) + T(10)

T(a) = 1 + 1 + 1 + 0 + 0 + 1 + 1 + 0 + 0 + T(7) + T(10)

T(a) = 1 + 1 + 1 + 0 + 0 + 1 + 1 + 0 + 0 + 1 + T(NULL) + T(NULL) + T(10)

T(a) = 1 + 1 + 1 + 0 + 0 + 1 + 1 + 0 + 0 + 1 + 0 + 0 + T(10)

T(a) = 6 + T(10)

T(a) = 6 + 1 + T(NULL) + T(14)

T(a) = 6 + 1 + 0 + 1 + T(NULL) + T(NULL)

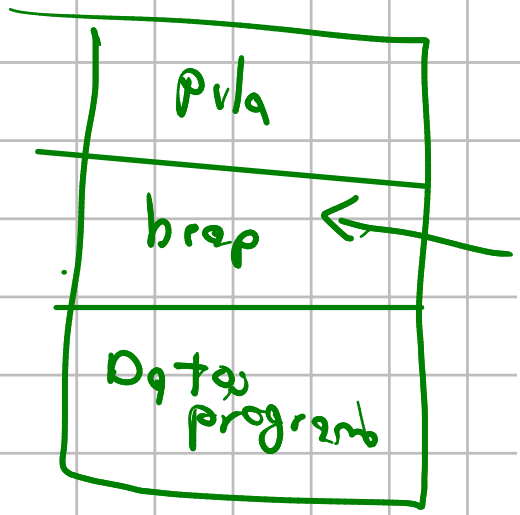
T(a) = 6 + 1 + 0 + 1 + 0 +0 = 8

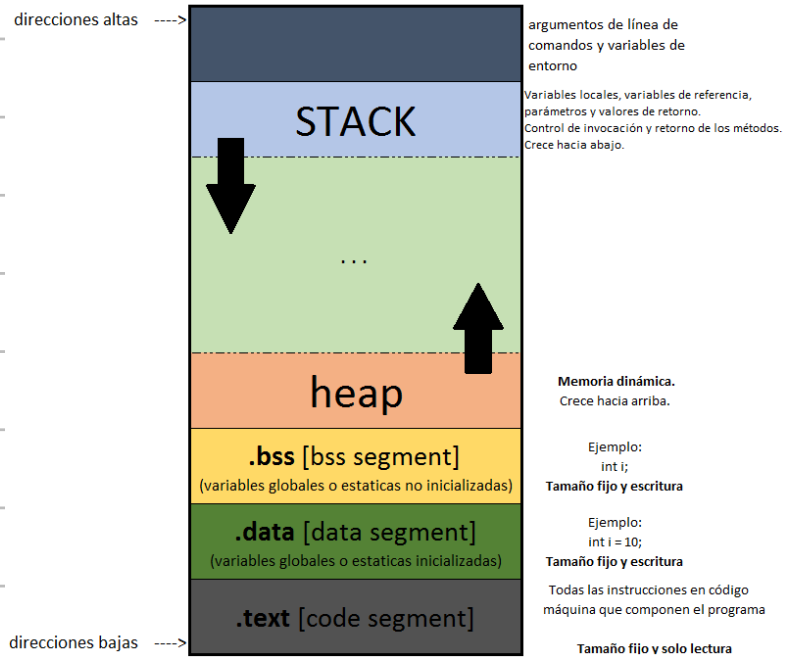
Lo que devuelve cada llamada II

Respuesta

$1 + 1 + 3 = 5$: el 3 con su subárbol izquierdo, de un nodo, y el derecho, de tres.

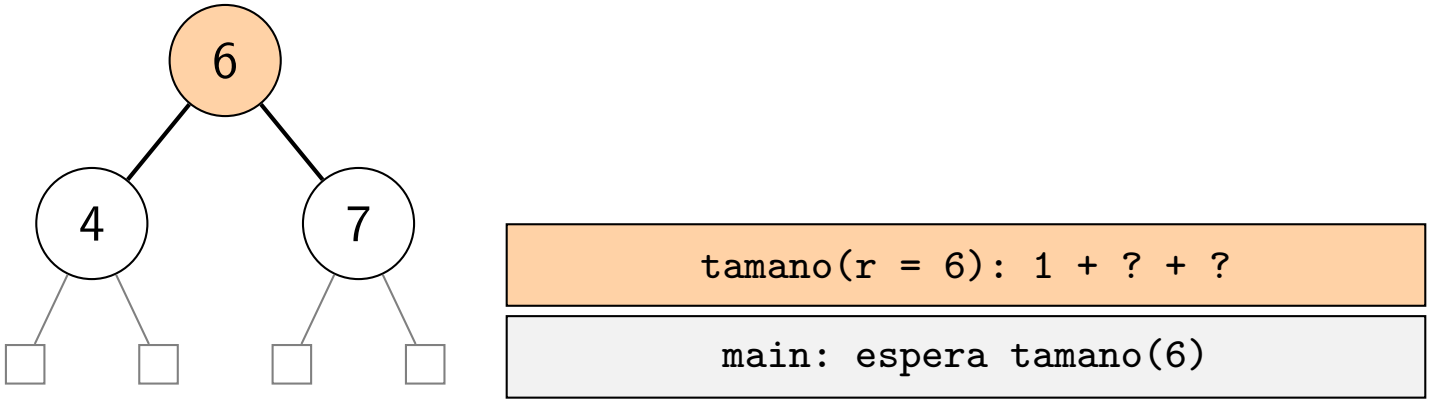
Para escribir `tamano` no hizo falta seguir todas las llamadas. Bastó suponer que la llamada sobre un subárbol, que tiene menos nodos, devuelve su tamaño, y ocuparse solo de combinar. Las llamadas se acaban porque cada una baja un nivel y tarde o temprano llega a un vacío, que responde sin llamar.


pila
heap
Data
program


direcciones altas ---->
argumentos de línea de comandos y variables de entorno
STACK
Variables locales, variables de referencia, parámetros y valores de retorno.
Control de invocación y retorno de los métodos.
Crece hacia abajo.
...
heap
Memoria dinámica.
Crece hacia arriba.
.bss [bss segment]
(variables globales o estaticas no inicializadas)
Ejemplo:
int i;
Tamaño fijo y escritura
.data [data segment]
(variables globales o estaticas inicializadas)
Ejemplo:
int i = 10;
Tamaño fijo y escritura
.text [code segment]
Todas las instrucciones en código máquina que componen el programa
Tamaño fijo y solo lectura
direcciones bajas ---->
https://netting.wordpress.com/ - https://netting.wordpress.com/ - https://netting.wordpress.com/

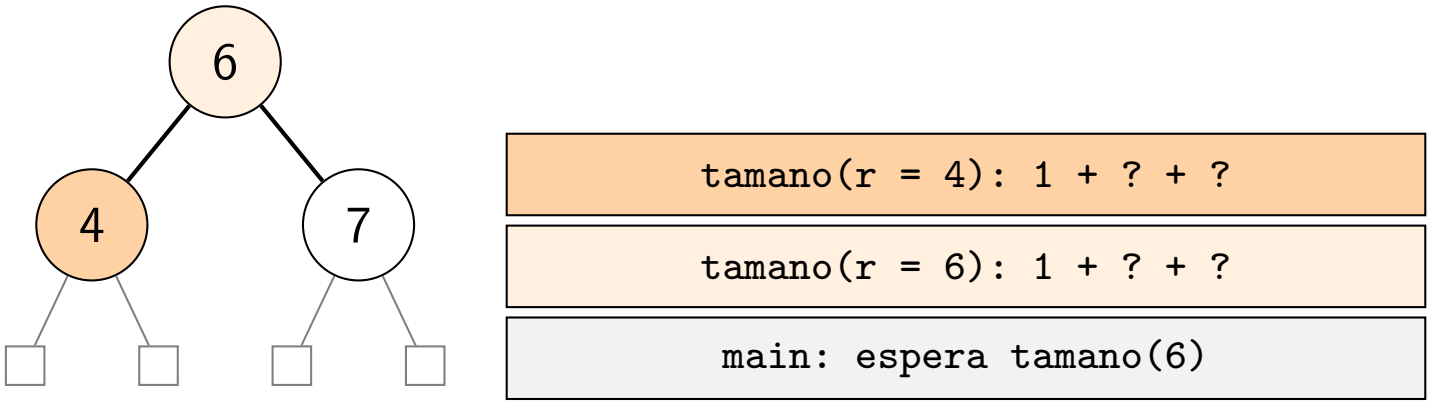
La pila de llamadas de `tamano(6)` I

Cada llamada abre un marco en la pila de llamadas con su `r` y lo que lleva sumado. Solo se ejecuta el marco de arriba (naranja); los de abajo esperan una respuesta. Se sigue `tamano` sobre el subárbol de 6, el más pequeño con dos hijos.



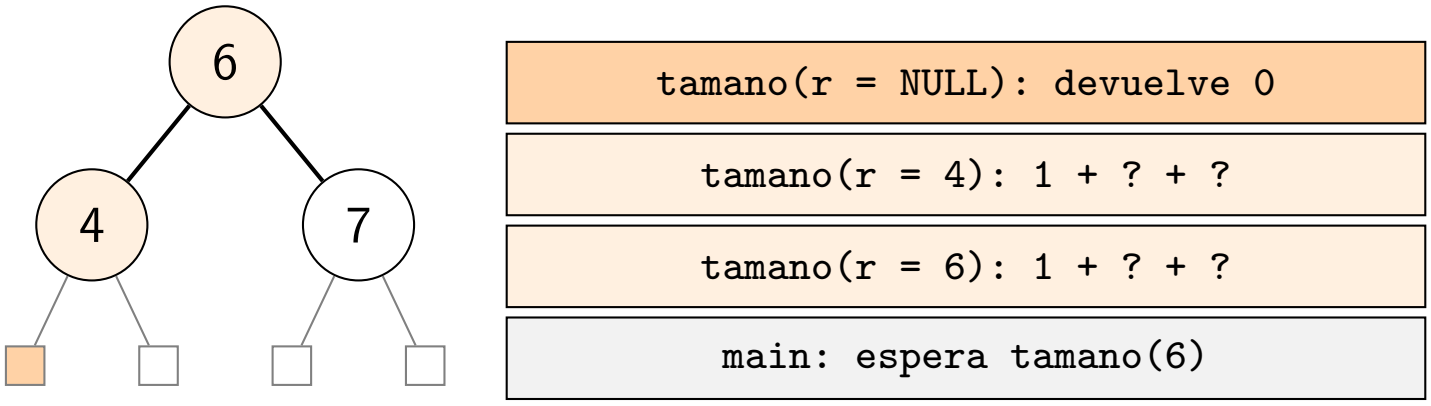
1. `main` llama `tamano(6)`. `r` no es `NULL`: caso recursivo. Para sumar necesita primero `tamano(4)`.

La pila de llamadas de `tamano(6)` II



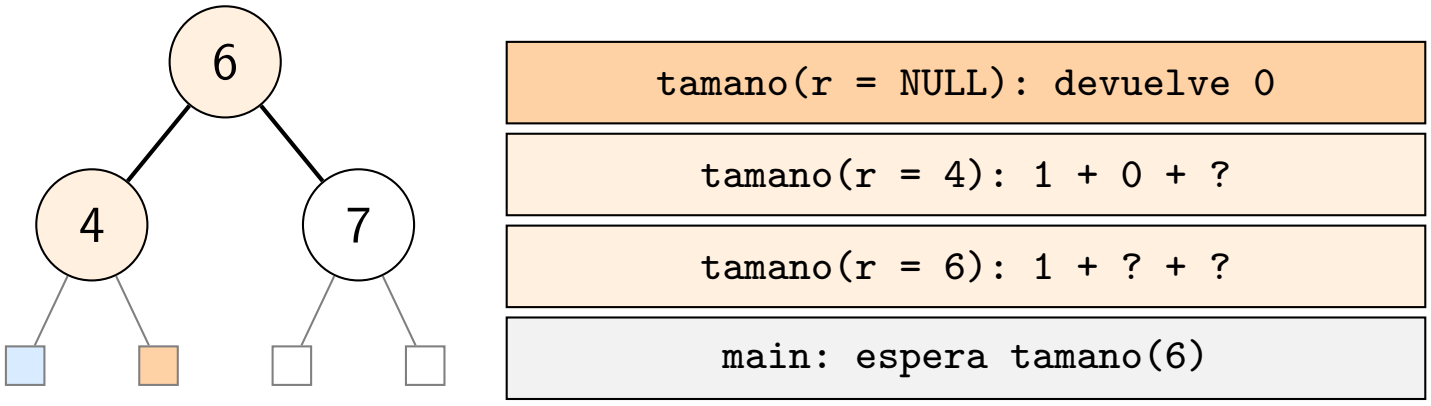
2. Entra el marco de 4, que también necesita a sus hijos. El de 6 queda esperando debajo.

La pila de llamadas de `tamano(6)` III



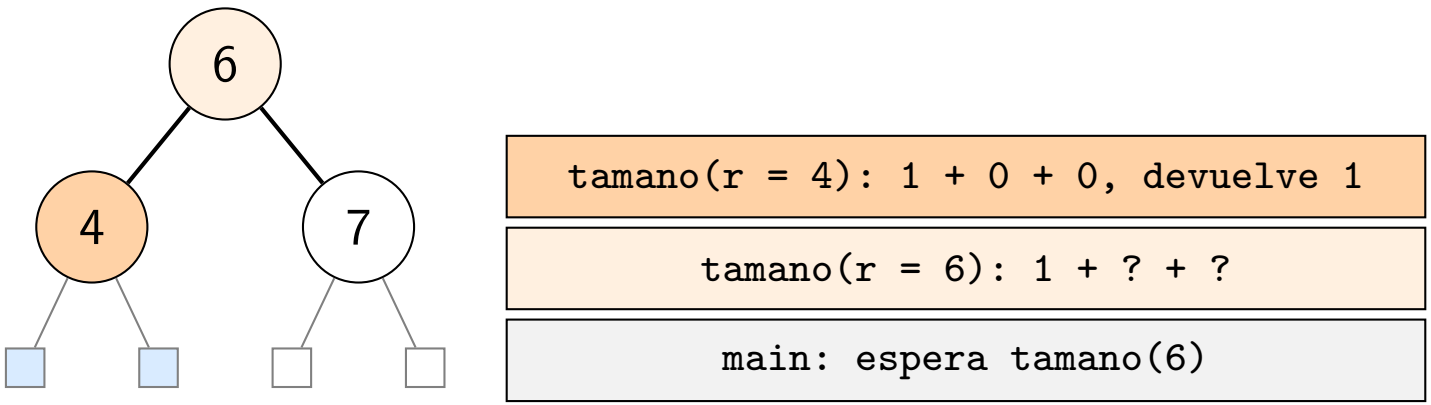
3. `r` es `NULL`: caso base. Devuelve 0 sin llamar a nadie, y su marco sale de la pila.

La pila de llamadas de `tamano(6)` IV



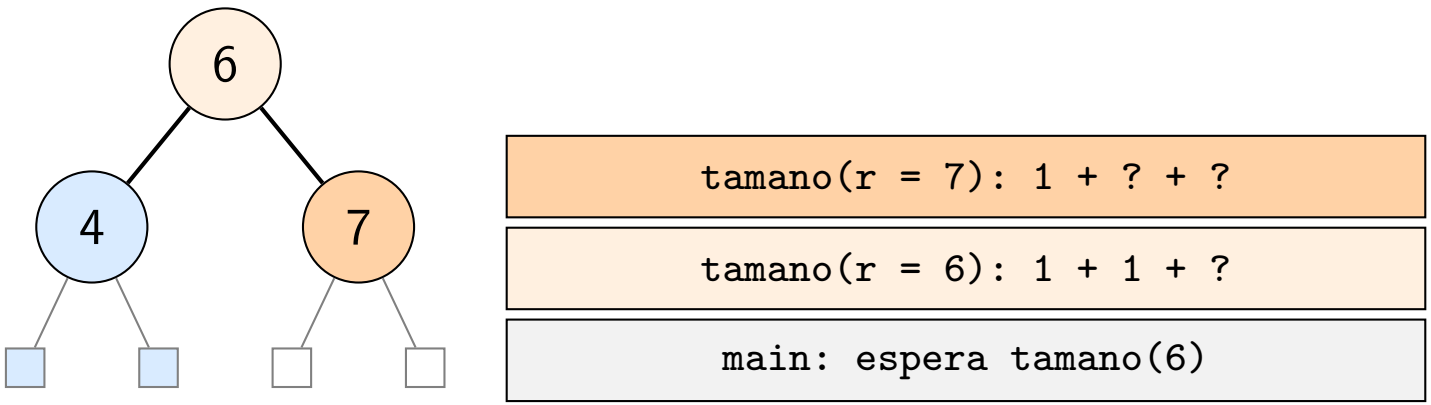
4. El 0 ocupa el primer hueco de 4. Ahora 4 llama con su hijo derecho, otro vacío, que también devuelve 0.

La pila de llamadas de `tamano(6)` V



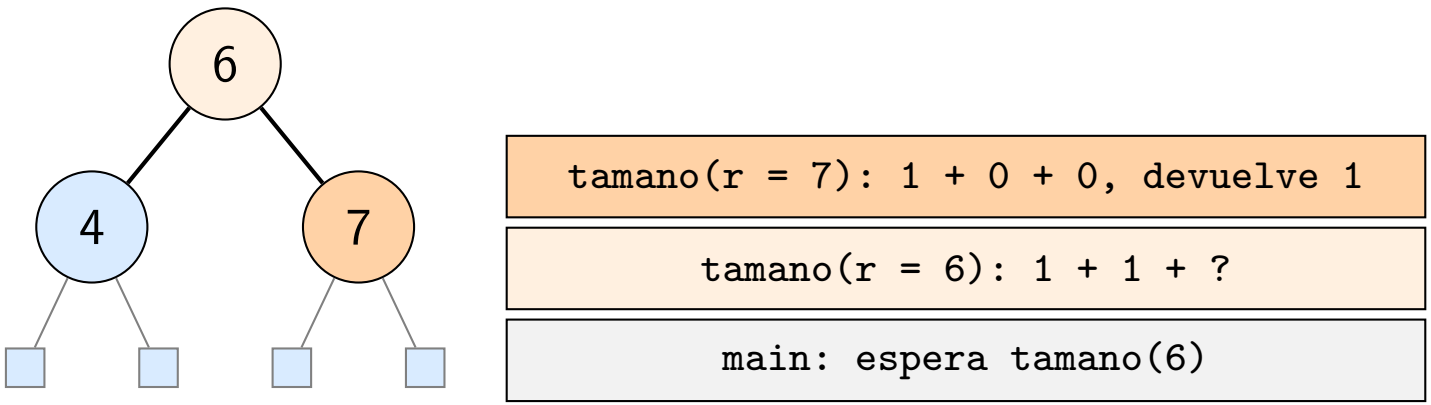
5. Con sus dos respuestas, 4 suma $1 + 0 + 0$ y devuelve 1. Su marco sale.

La pila de llamadas de `tamano(6)` VI



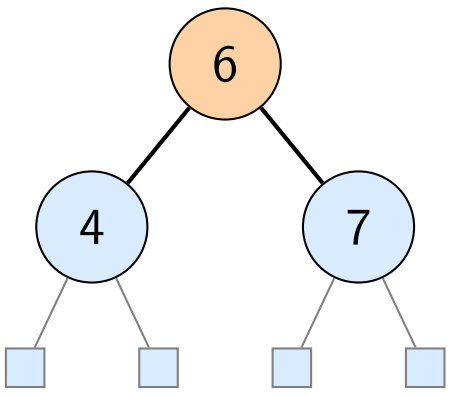
6. 6 guarda el 1 en su primer hueco y llama con su derecho: `tamano(7)`.

La pila de llamadas de `tamano(6)` VII



7. 7 repite lo de 4: llama con sus dos vacíos, recibe 0 y 0, y devuelve 1.

La pila de llamadas de `tamano(6)` VIII




```
tamano(r = 6): 1 + 1 + 1, devuelve 3
main: espera tamano(6)
```

8. 6 completa $1 + 1 + 1$ y devuelve 3 a `main`.

La pila de llamadas de `tamano(6)` IX

Lo que se vio en la pila

Hubo siete llamadas: tres sobre nodos y cuatro sobre vacíos. La pila llegó a tener tres marcos de `tamano` a la vez, el de 6, el de 4 y el de un vacío, que son los de un camino que baja desde la raíz.

La llamada más honda siempre cae en un vacío colgado del nodo más profundo. Sobre un árbol de altura h, la pila tiene a lo sumo $h + 2$ marcos de `tamano`: el espacio aparte es $O(h)$. En tiempo, cada nodo y cada vacío reciben una llamada que, sin contar las que hace, trabaja $\Theta(1)$; con n nodos eso da $\Theta(n)$.

altura I

Pregunta

Una hoja tiene altura 0, y la altura de un nodo es 1 más la mayor de las alturas de sus dos subárboles. ¿Cuánto debe valer la altura del vacío para que esa regla dé 0 en una hoja?

altura II

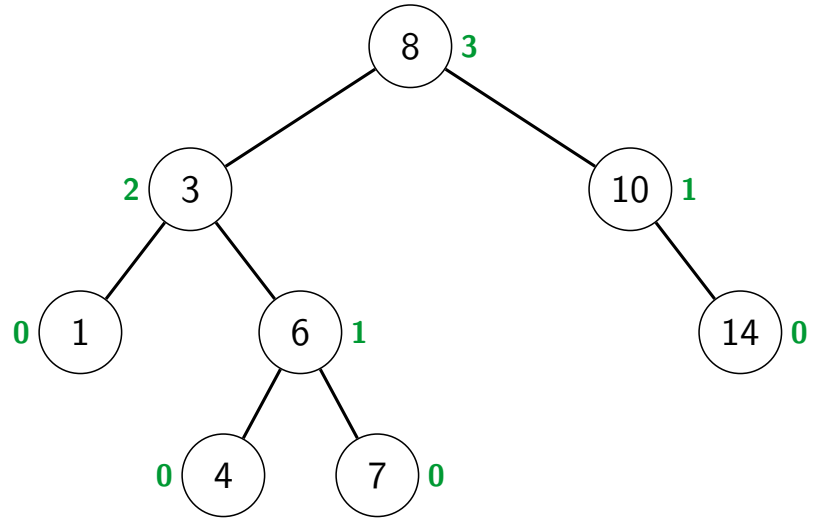
Respuesta

-1, porque en una hoja los dos subárboles son vacíos y $1 + \text{máx}(-1, -1) = 0$:

$$\mathit{altura}(T) = \begin{cases} -1 & \text{si } T \text{ es vacío} \\ 1 + \text{máx}(\mathit{altura}(I), \mathit{altura}(D)) & \text{si no} \end{cases}$$

altura III

```
// Aristas del camino mas largo
↪  de la raiz a una hoja.
// El arbol vacio tiene altura -1
↪  y una hoja, 0.
int altura(Nodo *r) {
  int res = -1;
  if (r != NULL) {
    int hi = altura(r->izq);
    int hd = altura(r->der);
    if (hi > hd) {
      res = 1 + hi;
    } else {
      res = 1 + hd;
    }
  }
  return res;
}
```



En verde, la altura de cada nodo. `altura(raiz)` da 3, por el camino 8, 3, 6, 4. Cuesta $\Theta(n)$ en tiempo y $O(h)$ de pila, igual que `tamano`.

Un caso base en el lugar equivocado I

Parece natural detenerse en la hoja, que tiene tamaño 1:

```
// Caso base en la hoja. Exige r != NULL.
int tamanoHoja(Nodo *r) {
  int res = 1;
  if (r->izq != NULL || r->der != NULL) {
    res = 1 + tamanoHoja(r->izq) + tamanoHoja(r->der);
  }
  return res;
}
```

Pregunta

¿Qué devuelve `tamanoHoja` sobre el subárbol de 6, sobre el de 3 y sobre el árbol completo?

Un caso base en el lugar equivocado II

Respuesta

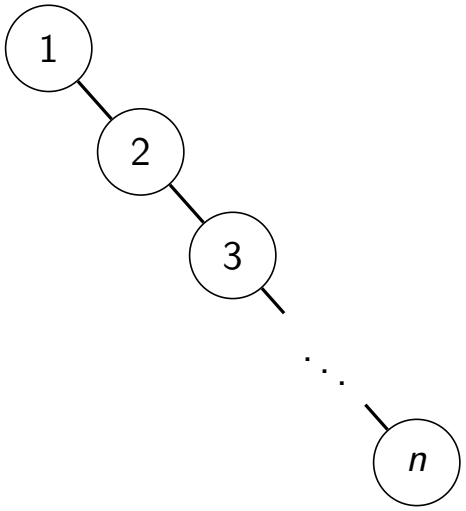
`tamanoHoja(seis): 3`
`tamanoHoja(tres): 5`
Con la raíz, el programa termina con `Segmentation fault`.

En los subárboles de 6 y de 3 cada nodo tiene cero o dos hijos, y la función acierta. El 10 tiene un solo hijo: el `if` entra y llama `tamanoHoja(NULL)` por la izquierda, y esa llamada lee `r->izq` con `r` nulo.

Sin caso base

Si a `tamano` se le quita el `if`, pasa lo mismo en la primera hoja: `tamano(NULL)` lee `NULL->izq`. El caso base de una función sobre árboles binarios es el vacío, y antes de leer `r->izq` o `r->dato` la función tiene que saber que `r` no es `NULL`.

Un árbol que es una lista I



`degenerado(n)` cuelga $1, 2, \ldots, n$, cada uno a la derecha del anterior. La altura es $n - 1$, y `altura` abre un marco por nodo antes de que el primero responda.

n	`altura(raiz)`
1 000	999
100 000	99 999
174 218	174 217
175 000	`Segmentation fault`

Un árbol que es una lista II

La pila de llamadas tiene tope

Linux da al programa una pila de llamadas de 8 MB (`ulimit -s` muestra 8192). Compilado con `g++ -Wall -Wextra`, cada marco de `altura` ocupa unos 48 bytes, así que caben unos 174 mil. Con 175 000 nodos el programa no se pone lento: se cae.

Un árbol que es una lista III

Los 175 000 nodos caben de sobra en memoria, porque se piden con `new`. Lo que se acaba es la pila de llamadas, que es mucho más pequeña. La salida es guardar lo pendiente en una pila propia, hecha también con nodos pedidos con `new`: los recorridos con pila explícita.

Para probarlo

```
g++ -Wall -Wextra desborde.cpp -o desborde
echo 175000 | ./desborde
```

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

Tres órdenes de visita I

Recorridos en profundidad (CLRS, §12.1; Thareja, §9.4)

- **Preorden**: la raíz, después el subárbol izquierdo y después el derecho. R I D
- **Inorden**: el subárbol izquierdo, la raíz y el derecho. I R D
- **Posorden**: el subárbol izquierdo, el derecho y al final la raíz. I D R

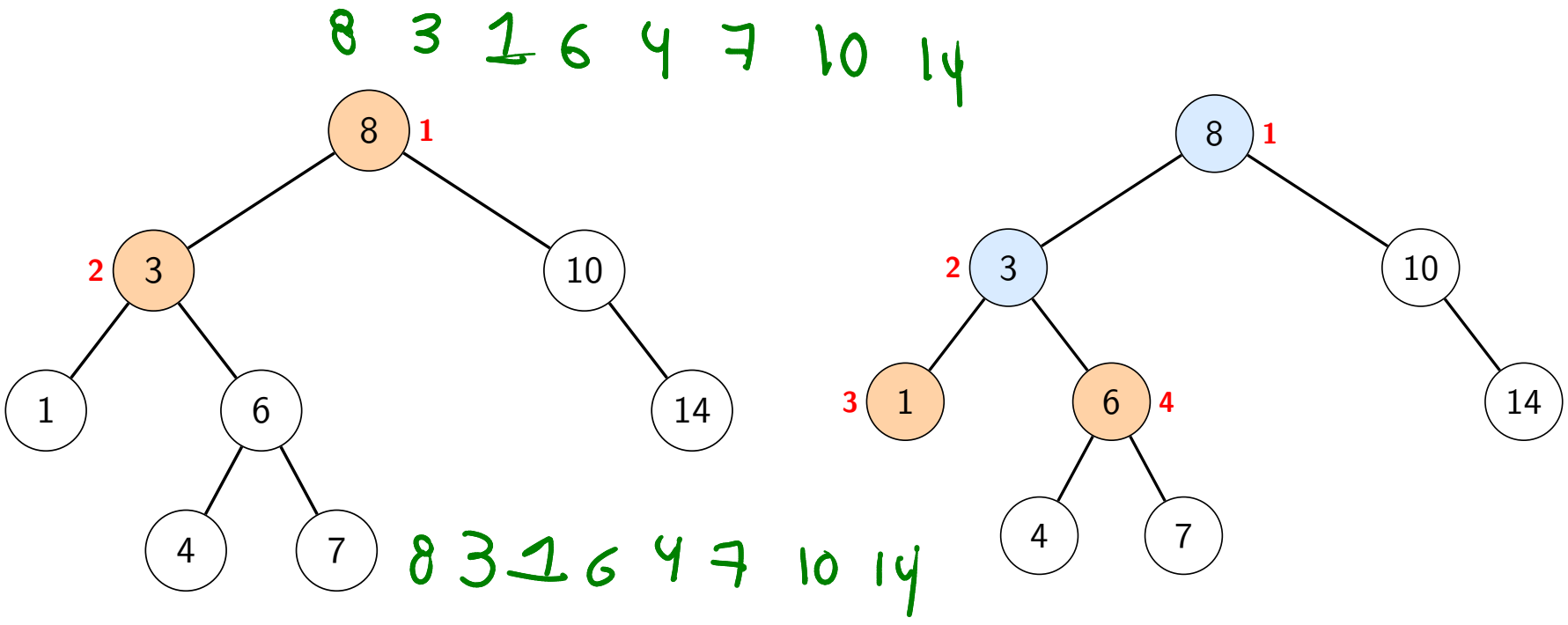
Los tres son recursivos y tienen el mismo caso base: sobre el vacío no se hace nada. Cambia una sola cosa, el momento en que se imprime la raíz: antes de bajar, entre las dos bajadas o después de las dos.

Preorden I

```c
// Raiz, subarbol izquierdo, subarbol derecho.
void preorden(Nodo *r) {
  if (r != NULL) {
    printf("%d ", r->dato);
    preorden(r->izq);
    preorden(r->der);
  }
}
```

El número rojo es el orden de visita; en naranja, los nodos que se imprimen en ese paso.

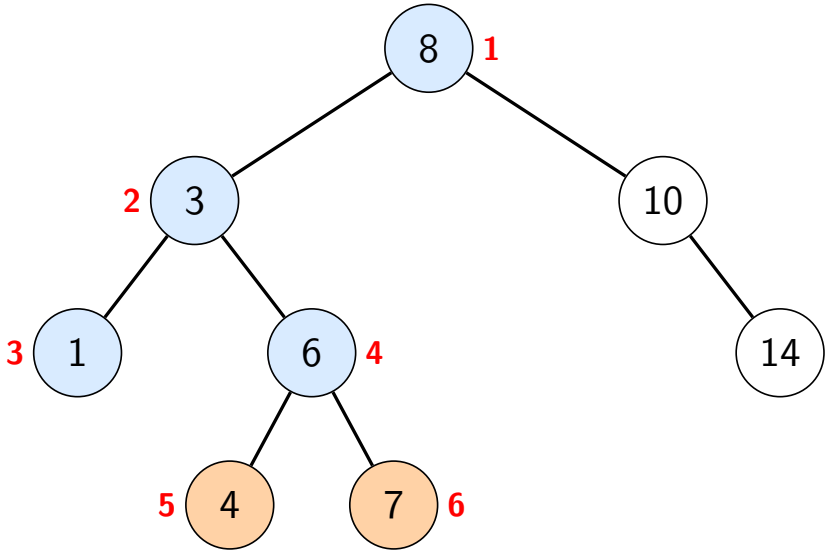
Preorden II



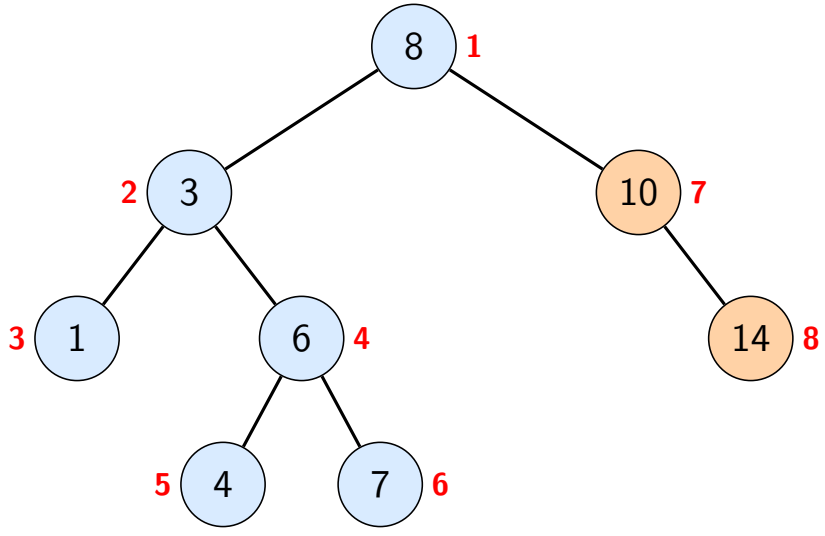
Se imprime 8 y se baja a la izquierda, al 3, que se imprime antes de bajar.
Salida: 8 3

1 es hoja: sus dos llamadas caen en vacíos.
Se vuelve a 3, que baja a la derecha: 6.
Salida: 8 3 1 6

Preorden III



Los hijos de 6, primero el izquierdo.
Salida: 8 3 1 6 4 7



Terminado el subárbol izquierdo de 8, sigue el derecho: 10 y, por su derecha, 14.
Salida: 8 3 1 6 4 7 10 14

Inorden I

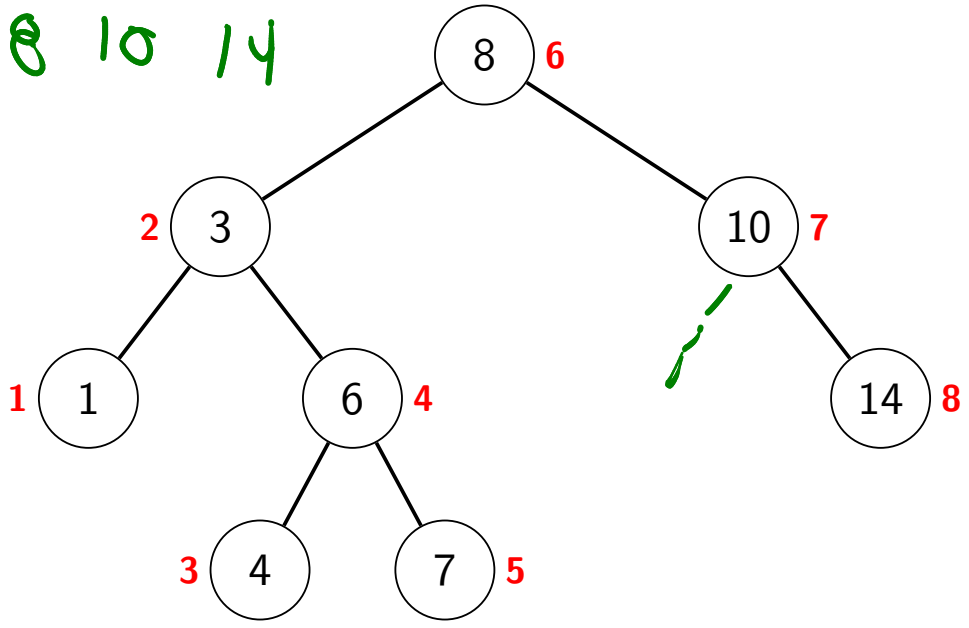
```c
// Subarbol izquierdo, raiz, subarbol derecho.
void inorden(Nodo *r) {
  if (r != NULL) {
    inorden(r->izq);
    printf("%d ", r->dato);
    inorden(r->der);
  }
}
```

Pregunta

¿En qué orden imprime `inorden` el árbol de ejemplo? ¿Cuál sale primero?

Inorden II

1 3 4 6 7 8 10 14



Respuesta

`1 3 4 6 7 8 10 14`. Sale primero el nodo de más a la izquierda: antes de imprimir, `inorden` baja por la izquierda hasta un vacío. El 10 se imprime antes que el 14 porque su subárbol izquierdo es vacío. La salida quedó ordenada de menor a mayor; al final de la sesión se vuelve sobre eso.

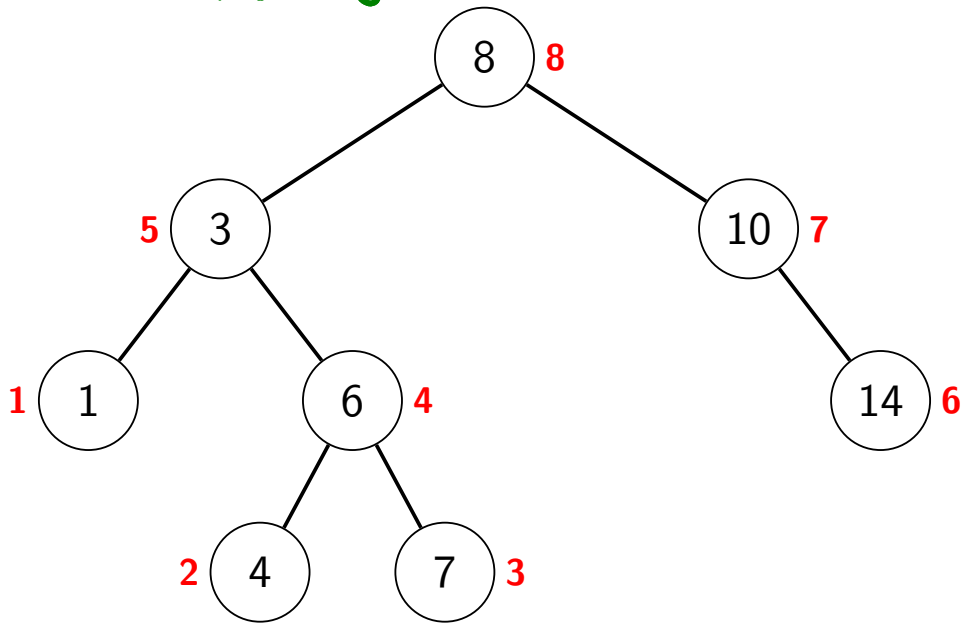
Posorden I

```c
// Subarbol izquierdo, subarbol derecho, raiz.
void posorden(Nodo *r) {
  if (r != NULL) {
    posorden(r->izq);
    posorden(r->der);
    printf("%d ", r->dato);
  }
}
```

Pregunta

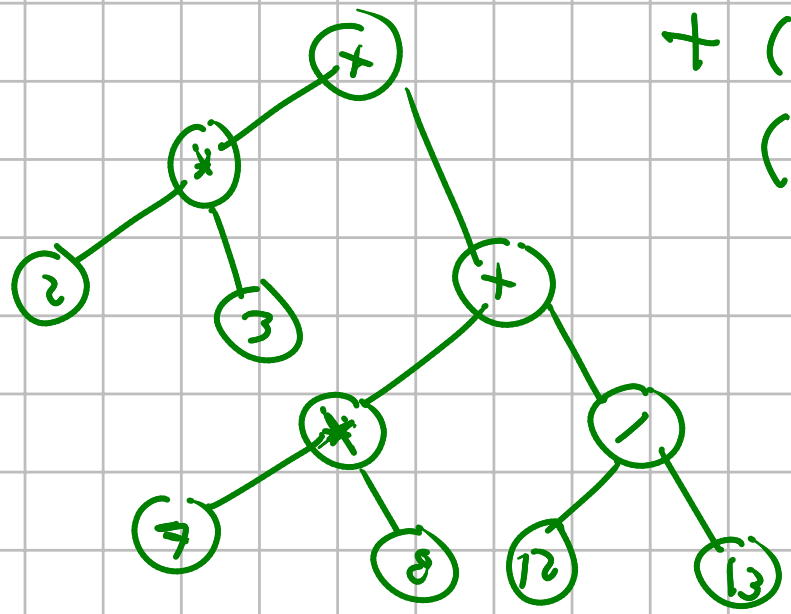
¿En qué orden imprime `posorden` el árbol de ejemplo? ¿Cuál sale de último?

Posorden II



Respuesta

1 4 7 6 3 14 10 8. Un nodo sale después de todo su subárbol, así que la raíz sale de última y las hojas van antes que sus padres.



+ (* 2 3)
(+ (* 7 8)
(/ 12 13)
)

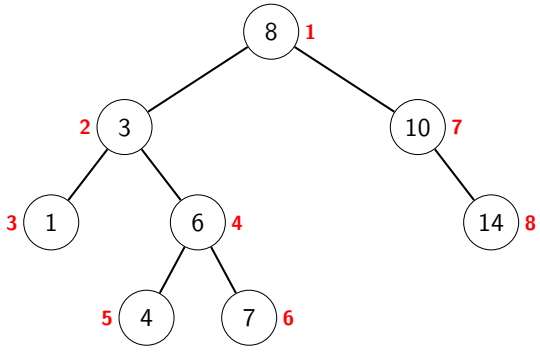
Prefix

(2 * 3) + (7 * 8) + (12 / 13) } Infix

((2 3 *) ((7 8 *) (12 13 /) +) +)

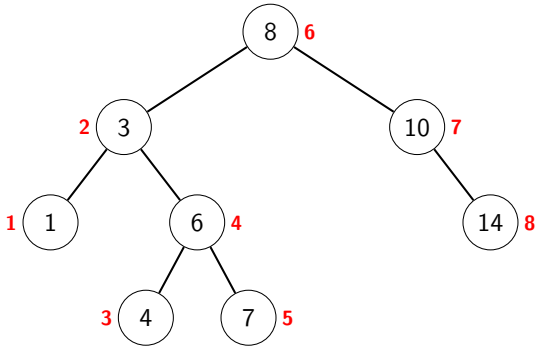
Los tres, lado a lado I

Preorden



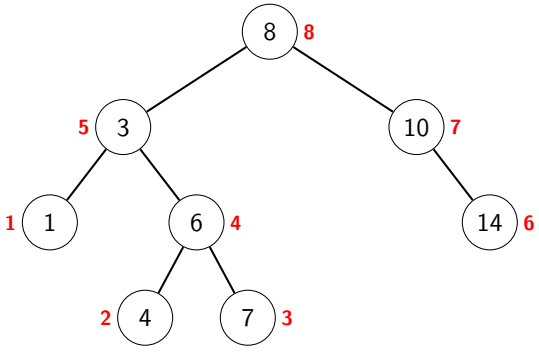
8 3 1 6 4 7 10 14

Inorden



1 3 4 6 7 8 10 14

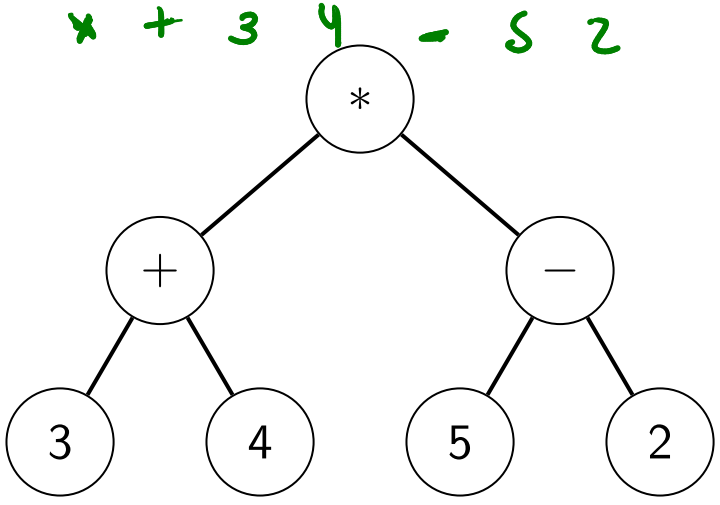
Posorden



1 4 7 6 3 14 10 8

Una forma de leerlos sin seguir llamadas: se camina por fuera del árbol, pegado a él, desde la izquierda de la raíz. Preorden anota cada nodo al pasar por su izquierda, inorden al pasar por debajo y posorden al pasar por su derecha. Los tres cuestan $\Theta(n)$ en tiempo, con un `printf` por nodo, y $O(h)$ de pila de llamadas.

Un árbol para $(3 + 4) * (5 - 2)$ I



En un árbol de expresión (Thareja, §9.2.5) las hojas son números y cada nodo interno es un operador con sus dos operandos como hijos. El nodo es el mismo `Nodo`: un operador se guarda como su carácter en el `int dato`.

(3 + 4) * (5 - 2)

```
Nodo *suma = crearNodo('+', crearNodo(3, NULL, NULL), crearNodo(4,
↪  NULL, NULL));
Nodo *resta = crearNodo('-', crearNodo(5, NULL, NULL),
↪  crearNodo(2, NULL, NULL));
Nodo *raiz = crearNodo('*', suma, resta);
```

(3 4 +) (5 2 -) *

Pregunta

¿Qué imprimen el preorden, el inorden y el posorden de este árbol?

Un árbol para $(3 + 4) * (5 - 2)$ II

Respuesta

```
prefija:  * + 3 4 - 5 2
infija:   ((3 + 4) * (5 - 2))
postfija: 3 4 + 5 2 - *
```

El preorden da la notación prefija, el inorden la infija y el posorden la postfija, la que evalúa una pila de operandos. La prefija y la postfija no necesitan paréntesis. La infija sí: sin ellos, `3 + 4 * 5 - 2` se agruparía de otra manera, y por eso `infija` pone un par alrededor de cada operación.

Un árbol para $(3 + 4) * (5 - 2)$ III

```
// Exige r != NULL.
bool esHoja(Nodo *r) {
  return r->izq == NULL && r->der == NULL;
}

// Imprime un nodo: el numero si es hoja, el operador si no.
void imprimirNodo(Nodo *r) {
  if (esHoja(r)) {
    printf("%d ", r->dato);
  } else {
    printf("%c ", r->dato);
  }
}
```

Un árbol para $(3 + 4) * (5 - 2)$ IV

`prefija` y `postfija` son `preorden` y `posorden` con `imprimirNodo` en el lugar del `printf`:

```
// Preorden: el operador antes que sus operandos.
void prefija(Nodo *r) {
  if (r != NULL) {
    imprimirNodo(r);
    prefija(r->izq);
    prefija(r->der);
  }
}
```

```
// Posorden: los operandos antes que su operador.
void postfija(Nodo *r) {
  if (r != NULL) {
    postfija(r->izq);
    postfija(r->der);
    imprimirNodo(r);
  }
}
```

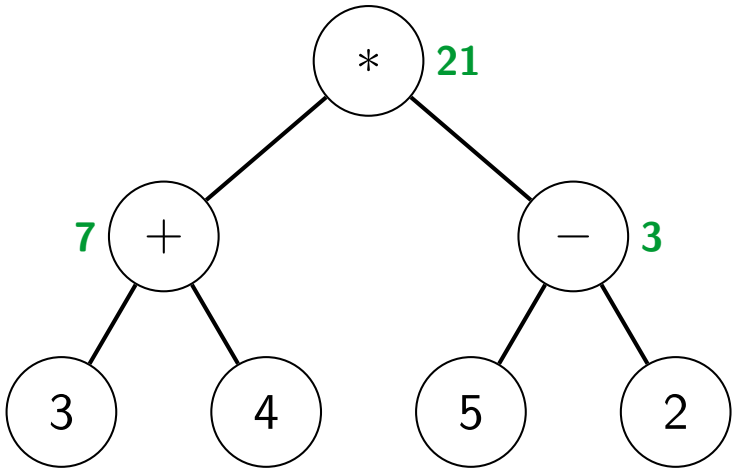
Un árbol para $(3 + 4) * (5 - 2)$ V

```
// Inorden con un parentesis alrededor de cada operacion. Exige r !=
↪  NULL.
void infija(Nodo *r) {
  if (esHoja(r)) {
    printf("%d", r->dato);
  } else {
    printf("(");
    infija(r->izq);
    printf(" %c ", r->dato);
    infija(r->der);
    printf(")");
  }
}
```

`infija` es un inorden que abre un paréntesis antes de bajar a la izquierda y lo cierra al volver de la derecha.

Evaluar el árbol I

```
// Posorden: se evaluan los dos
↪  operandos y despues se opera.
↪  Exige r != NULL.
int evaluar(Nodo *r) {
  int res = r->dato;
  if (!esHoja(r)) {
    int a = evaluar(r->izq);
    int b = evaluar(r->der);
    if (r->dato == '+') {
      res = a + b;
    } else if (r->dato == '-') {
      res = a - b;
    } else if (r->dato == '*') {
      res = a * b;
    } else {
      res = a / b;
    }
  }
  return res;
}
```



`evaluar` es un posorden: primero los dos operandos, después el operador. En verde, lo que devuelve cada nodo interno.

`valor:` 21

Evaluar el árbol II

Cuesta $\Theta(n)$ en tiempo y $O(h)$ de pila de llamadas.

Lo que `evaluar` da por hecho

- Que `r` no es `NULL` y que cada operador tiene sus dos hijos: un árbol de expresión no tiene nodos internos con un solo hijo. Por eso el caso base es la hoja, y no el vacío.
- Que no hay división por cero: con `/` y un divisor que vale 0, el programa se cae.
- Un nodo se reconoce como operador por tener hijos, no por su valor: una hoja con el número 42, que es el código de `*`, se imprime y se evalúa como 42.

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

preordenPila I

```
// Se saca un nodo, se imprime y se apilan sus hijos:
// el derecho primero, para que el izquierdo salga antes.
void preordenPila(Nodo *r) {
  Pila p;
  if (r != NULL) {
    p.apilar(r);
  }
  while (!p.vacia()) {
    Nodo *x = p.tope();
    p.desapilar();
    printf("%d ", x->dato);
    if (x->der != NULL) {
      p.apilar(x->der);
    }
    if (x->izq != NULL) {
      p.apilar(x->izq);
    }
  }
}
```

`preordenPila` II

La pila guarda los subárboles que faltan por visitar. El derecho entra antes que el izquierdo para que el izquierdo salga primero, como en la recursiva. Sobre el árbol de ejemplo, con el tope a la izquierda:

Vuelta	Saca e imprime	Apila	Pila después	Salida
inicio		8	8	
1	8	10, 3	3 10	8
2	3	6, 1	1 6 10	8 3
3	1		6 10	8 3 1
4	6	7, 4	4 7 10	8 3 1 6
5	4		7 10	8 3 1 6 4
6	7		10	8 3 1 6 4 7
7	10	14	14	8 3 1 6 4 7 10
8	14		vacía	8 3 1 6 4 7 10 14

La salida es la del preorden recursivo. Una vuelta por nodo: $\Theta(n)$. La pila guarda a lo sumo un hijo derecho pendiente por nivel más el nodo actual: $O(h)$.

inordenPila I

```
// Se baja por la izquierda apilando; al llegar a NULL se saca
// el tope, se imprime y se sigue por su subarbol derecho.
void inordenPila(Nodo *r) {
  Pila p;
  Nodo *actual = r;
  while (actual != NULL || !p.vacia()) {
    if (actual != NULL) {
      p.apilar(actual);
      actual = actual->izq;
    } else {
      actual = p.tope();
      p.desapilar();
      printf("%d ", actual->dato);
      actual = actual->der;
    }
  }
}
```

`inordenPila` II

Cada vuelta hace una de dos cosas. Si `actual` es un nodo, lo apila y baja a la izquierda. Si es `NULL`, el tope es un nodo cuyo subárbol izquierdo ya se imprimió: se saca, se imprime y se sigue por su derecho.

inordenPila III

Vuelta	actual	Hace	Pila después	Imprime
1	8	apila, baja a 3	8	
2	3	apila, baja a 1	3 8	
3	1	apila, baja a NULL	1 3 8	
4	NULL	saca 1, va a su derecho, NULL	3 8	1
5	NULL	saca 3, va a 6	8	3
6	6	apila, baja a 4	6 8	
7	4	apila, baja a NULL	4 6 8	
8	NULL	saca 4, va a NULL	6 8	4

`inordenPila` IV

Vuelta	`actual`	Hace	Pila después	Imprime
9	`NULL`	saca 6, va a 7	8	6
10	7	apila, baja a `NULL`	7 8	
11	`NULL`	saca 7, va a `NULL`	8	7
12	`NULL`	saca 8, va a 10	vacía	8
13	10	apila, baja a `NULL`	10	
14	`NULL`	saca 10, va a 14	vacía	10
15	14	apila, baja a `NULL`	14	
16	`NULL`	saca 14, va a `NULL`	vacía	14

En la vuelta 12 la pila queda vacía, pero `actual` es 10: por eso la condición del `while` lleva un `||`. Son $2n = 16$ vueltas, cada nodo entra y sale una vez: $\Theta(n)$. La pila guarda el camino de nodos que esperan su turno: $O(h)$.

posordenPila I

```
void posordenPila(Nodo *r) {
  Pila p1;
  Pila p2;
  if (r != NULL) {
    p1.apilar(r);
  }
  while (!p1.vacia()) {
    Nodo *x = p1.tope();
    p1.desapilar();
    p2.apilar(x);
    if (x->izq != NULL) {
      p1.apilar(x->izq);
    }
    if (x->der != NULL) {
      p1.apilar(x->der);
    }
  }
  while (!p2.vacia()) {
    printf("%d ", p2.tope()->dato);
    p2.desapilar();
  }
}
```

posordenPila II

Vuelta	Saca de p1	Apila en p1	p1 después	p2 después
inicio		8	8	vacía
1	8	3, 10	10 3	8
2	10	14	14 3	10 8
3	14		3	14 10 8
4	3	1, 6	6 1	3 14 10 8
5	6	4, 7	7 4 1	6 3 14 10 8
6	7		4 1	7 6 3 14 10 8
7	4		1	4 7 6 3 14 10 8
8	1		vacía	1 4 7 6 3 14 10 8

p2 recibe los nodos en el orden raíz, derecho, izquierdo: 8 10 14 3 6 7 4 1. Al vaciarla sale al revés, 1 4 7 6 3 14 10 8, que es el posorden.

posordenPila III

El precio de la segunda pila

p2 guarda los n nodos antes de imprimir el primero: el espacio aparte es $\Theta(n)$, contra $O(h)$ de la recursiva. El tiempo sigue en $\Theta(n)$.

De vuelta al árbol degenerado I

```
int alturaPila(Nodo *r) {
  int res = -1;
  Pila p;
  Nodo *actual = r;
  Nodo *ultimo = NULL;  // ultimo nodo que salio de la pila
  while (actual != NULL || !p.vacia()) {
    if (actual != NULL) {
      p.apilar(actual);
      if (p.tamano() - 1 > res) {
        res = p.tamano() - 1;
      }
      actual = actual->izq;
    } else if (p.tope()->der != NULL && p.tope()->der != ultimo) {
      actual = p.tope()->der;
    } else {
      ultimo = p.tope();
      p.desapilar();
    }
  }
  return res;
}
```

De vuelta al árbol degenerado II

`alturaPila` es un posorden con una sola pila. Un nodo sale de la pila cuando su derecho ya se recorrió, y `ultimo` recuerda cuál salió para no bajar dos veces por la misma rama. La pila guarda el camino de la raíz al nodo actual, como los marcos de `altura`, y su tamaño menos uno es la profundidad del nodo.

Con $n = 200\,000$

```
n = 200000
alturaPila: 199999
```

y después `altura`, la recursiva, termina con `Segmentation fault`.

La pila propia guarda los mismos 200 000 nodos del camino, pero en memoria pedida con `new`. `liberar` también es recursiva y se caería con ese árbol; `desborde.cpp` libera con `liberarPila`, que apila los hijos de un nodo antes de borrarlo.

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

Liberar el árbol I

Pregunta

`liberar(r)` debe hacer `delete` de cada nodo. ¿En cuál de los tres órdenes? ¿Qué pasa si borra la raíz antes de bajar a los hijos?

Liberar el árbol II

Respuesta

En posorden: los hijos antes que el padre.

```
// Libera en posorden: los hijos antes que el padre.
void liberar(Nodo *r) {
  if (r != NULL) {
    liberar(r->izq);
    liberar(r->der);
    delete r;
  }
}
```

Liberar el árbol III

Un `delete` por nodo: $\Theta(n)$ en tiempo y $O(h)$ de pila de llamadas.

Borrar primero

En preorden, `delete r` va antes de `liberar(r->izq)`, y esa llamada lee un campo de memoria ya devuelta. El resultado no está definido: puede funcionar en una prueba y fallar en la siguiente. Si se quiere borrar primero, hay que guardar `r->izq` y `r->der` en variables antes del `delete`, que es lo que hace `liberarPila` con su pila.

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

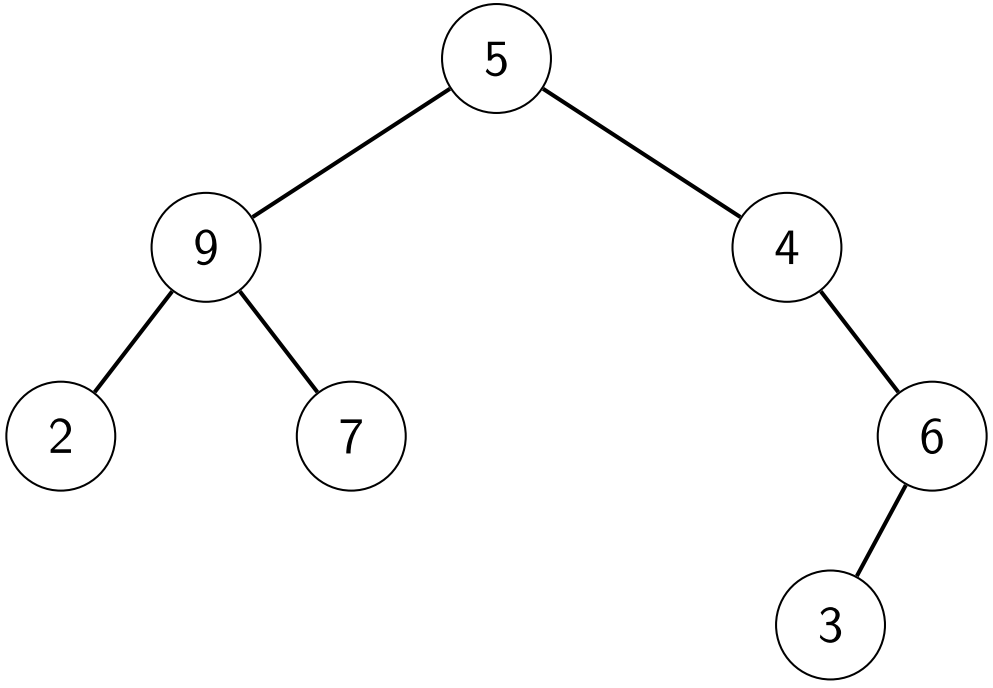
8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

El árbol de los ejercicios I



Siete nodos. Cada ejercicio se resuelve con dos preguntas antes de escribir código: ¿cuál es el caso base?, ¿qué se hace con lo que devuelven los hijos? Después se calcula lo que da en este árbol.

Ejercicio en parejas: contar hojas I

Pregunta

Escriba `int contarHojas(Nodo *r)`. ¿Cuál es el caso base? ¿Qué se hace con lo que devuelven los hijos? ¿Cuánto da en el árbol de los ejercicios?

Ejercicio en parejas: contar hojas II

Respuesta

Hay dos casos base: el vacío no tiene hojas y una hoja cuenta 1. En un nodo interno se suman las hojas de los dos subárboles; el nodo mismo no suma. En el árbol de los ejercicios da 3 (2, 7 y 3), y en el de ejemplo, 4. $\Theta(n)$ en tiempo y $O(h)$ de pila.

Ejercicio en parejas: contar hojas III

```
// Una hoja cuenta 1; un nodo interno, lo que cuenten sus dos
↪  subarboles.
int contarHojas(Nodo *r) {
  int res = 0;
  if (r != NULL) {
    if (r->izq == NULL && r->der == NULL) {
      res = 1;
    } else {
      res = contarHojas(r->izq) + contarHojas(r->der);
    }
  }
  return res;
}
```

Ejercicio en parejas: sumar los datos I

Pregunta

Escriba `int sumar(Nodo *r)`, la suma de todos los datos. ¿Caso base? ¿Qué se hace con lo que devuelven los hijos? ¿Cuánto da en el árbol de los ejercicios?

Ejercicio en parejas: sumar los datos II

Respuesta

El vacío suma 0. Un nodo suma su dato más lo que devuelven sus dos subárboles: es `tamano` con `r->dato` en el lugar del 1. En el árbol de los ejercicios da 36 y en el de ejemplo, 53. $\Theta(n)$ en tiempo y $O(h)$ de pila.

```
int sumar(Nodo *r) {
  int res = 0;
  if (r != NULL) {
    res = r->dato + sumar(r->izq) + sumar(r->der);
  }
  return res;
}
```

Ejercicio en parejas: el máximo I

Pregunta

Escriba `int maximo(Nodo *r)`. ¿Qué debería devolver el máximo de un árbol vacío? ¿Cuál es entonces el caso base? ¿Cuánto da en el árbol de los ejercicios?

Ejercicio en parejas: el máximo II

Respuesta

El vacío no tiene máximo, y devolver 0 fallaría si todos los datos fueran negativos. `maximo` exige `r != NULL` y baja solo a los subárboles que no son vacíos; el caso base es el nodo sin hijos, que devuelve su dato. Con los hijos, se compara el dato con lo que devuelve cada uno.

En el árbol de los ejercicios da 9, que no es la raíz, ni una hoja, ni el nodo de más a la derecha. En el de ejemplo da 14. $\Theta(n)$ en tiempo y $O(h)$ de pila.

Ejercicio en parejas: el máximo III

```c
// exige r != NULL; solo baja a los subarboles que no son vacios
int maximo(Nodo *r) {
  assert(r != NULL);
  int res = r->dato;
  if (r->izq != NULL) {
    int mi = maximo(r->izq);
    if (mi > res) {
      res = mi;
    }
  }
  if (r->der != NULL) {
    int md = maximo(r->der);
    if (md > res) {
      res = md;
    }
  }
  return res;
}
```

Ejercicio en parejas: nodos del nivel k I

Pregunta

Escriba `int contarNivel(Nodo *r, int k)`, cuántos nodos tienen profundidad k. ¿Qué cambia en `k` al bajar a un hijo? ¿Cuáles son los casos base? ¿Cuánto da en el árbol de los ejercicios para $k = 0, 1, 2, 3, 4$?

Ejercicio en parejas: nodos del nivel k II

Respuesta

El nivel k de `r` es el nivel $k-1$ de cada hijo: aquí la información también baja, por el parámetro. Casos base: el vacío da 0, y con $k = 0$ el nodo mismo cuenta 1. En el árbol de los ejercicios: 1, 2, 3, 1 y 0. No baja más allá del nivel k: $O(n)$ en tiempo y $O(\text{mín}(k, h))$ de pila.

Ejercicio en parejas: nodos del nivel k III

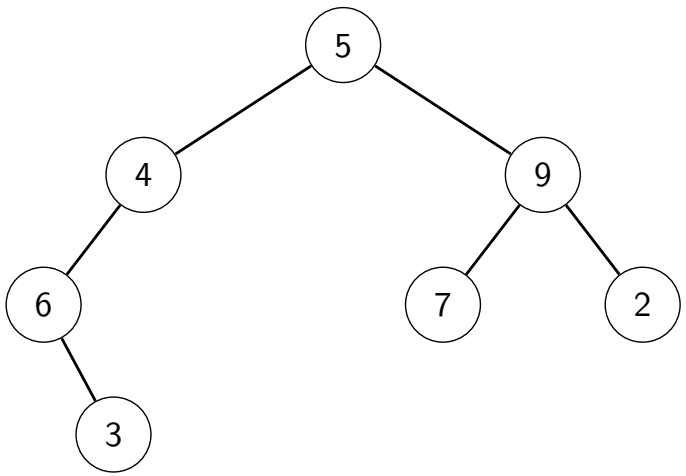
```
// Nodos a profundidad k: el nivel k de r es el nivel k - 1 de sus
↪  hijos.
int contarNivel(Nodo *r, int k) {
  int res = 0;
  if (r != NULL) {
    if (k == 0) {
      res = 1;
    } else {
      res = contarNivel(r->izq, k - 1) + contarNivel(r->der, k - 1);
    }
  }
  return res;
}
```

Ejercicio en parejas: el espejo I

Pregunta

Escriba `void espejo(Nodo *r)`, que intercambia el izquierdo y el derecho en cada nodo sin crear nodos. ¿Caso base? ¿Qué se hace en cada nodo? Dibuje el espejo del árbol de los ejercicios y dé su preorden y su inorden.

Ejercicio en parejas: el espejo II



Respuesta

El vacío no se toca. En un nodo se intercambian los dos punteros y se aplica `espejo` a cada hijo. Preorden 5 4 6 3 9 7 2; inorden 6 3 4 5 7 9 2, el inorden original al revés.

Ejercicio en parejas: el espejo III

```
// Intercambia los hijos de cada nodo, sin crear nodos nuevos.
void espejo(Nodo *r) {
  if (r != NULL) {
    Nodo *temp = r->izq;
    r->izq = r->der;
    r->der = temp;
    espejo(r->izq);
    espejo(r->der);
  }
}
```

$\Theta(n)$ en tiempo y $O(h)$ de pila. Aplicado dos veces deja el árbol como estaba.

Ejercicio en parejas: dos árboles iguales I

Pregunta

Escriba `bool iguales(Nodo *a, Nodo *b)`: misma forma y mismos datos en la misma posición. Ahora la función recibe dos árboles, ¿cuántos casos base hay? Si en el árbol de los ejercicios el 3 pasa a ser hijo derecho del 6, ¿cambia el preorden? ¿Qué devuelve `iguales`?

Ejercicio en parejas: dos árboles iguales II

Respuesta

Dos vacíos son iguales; un vacío y un nodo, no. Con dos nodos se pide el mismo dato, izquierdos iguales y derechos iguales. Al mover el 3, el preorden sigue siendo `5 9 2 7 4 6 3`, pero `iguales` devuelve falso y el inorden lo delata: `2 9 7 5 4 6 3`. Un solo recorrido no determina el árbol.

Ejercicio en parejas: dos árboles iguales III

```
// Misma forma y mismos datos en la misma posicion.
bool iguales(Nodo *a, Nodo *b) {
  bool res = false;
  if (a == NULL && b == NULL) {
    res = true;
  } else if (a != NULL && b != NULL) {
    res = a->dato == b->dato && iguales(a->izq, b->izq) &&
    ↪  iguales(a->der, b->der);
  }
  return res;
}
```

El && deja de comparar en la primera diferencia: $O(n)$ en tiempo y $O(h)$ de pila.

Propuestos para la casa I

1. **Un solo hijo.** Escriba `contarUnHijo(r)`, los nodos con exactamente un hijo. ¿Cuánto da en los dos árboles de la sesión?
2. **¿Dónde está?** Escriba `profundidad(r, x)`, la profundidad del nodo con dato `x`, o -1 si no está.
3. **El camino más pesado.** La mayor suma de los datos de un camino que va de la raíz a una hoja.
4. **Árbol lleno.** `esLleno(r)` dice si cada nodo tiene cero o dos hijos. ¿Qué tiene que ver con `tamanoHoja`?
5. **Por niveles.** Escriba `imprimirNivel(r, k)` con la idea de `contarNivel`, y úsela en un `while` con k de 0 a `altura(r)`. ¿Cuánto cuesta en un árbol degenerado?

Pistas para los propuestos I

1. Como `contarHojas`, con la condición de exactamente un hijo para contar 1 más lo de los subárboles. Da 1 en el de ejemplo (el 10) y 2 en el de los ejercicios (el 4 y el 6).
2. Dos casos base: el vacío da -1 y el nodo con `x` da 0. Si un subárbol devuelve $d \geq 0$, la respuesta es $d + 1$.
3. El caso base es la hoja, como en `maximo`. Da 32 en el de ejemplo (8, 10, 14) y 21 en el de los ejercicios (5, 9, 7).
4. El vacío es lleno; un nodo con un solo hijo, no. Son justamente los árboles donde `tamanoHoja` no se cae.
5. El caso base con $k = 0$ imprime en vez de contar. Cada nivel vuelve a bajar desde la raíz: sume lo que cuesta cada vuelta del `while` cuando la altura es $n - 1$.

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

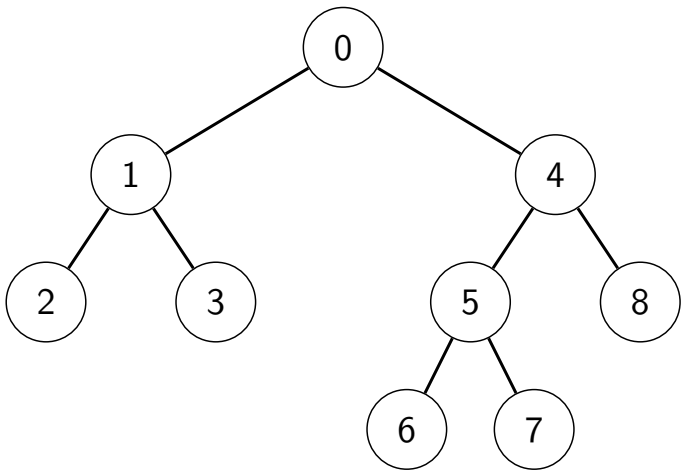
Para el juez: Aizu ALDS1_7_C, Tree Walk I

Aizu ALDS1_7_C, Tree Walk (Aizu Online Judge)
Enunciado:
`https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_7_C`
Envío: `https://vjudge.net/problem/Aizu-ALDS1_7_C`

Qué pide

Un árbol binario de n nodos, $1 \leq n \leq 25$, con ID de 0 a $n-1$. Cada una de las n líneas trae `id left right`, con -1 donde falta un hijo. Se imprimen los recorridos preorden, inorden y posorden, cada uno bajo una línea con su nombre (`Preorder`, `Inorder`, `Postorder`).

Para el juez: Aizu ALDS1_7_C, Tree Walk II



¿Entendimos el problema?

El ejemplo del enunciado es este árbol, y su preorden es 0 1 2 3 4 5 6 7 8. ¿Por qué el inorden empieza por 2 y el posorden termina en 0?

El 2 es el nodo de más a la izquierda: el inorden baja por 0 y 1 hasta un vacío antes de imprimir. El 0 es la raíz, y en posorden la raíz sale de última.

Para el juez: Aizu ALDS1_7_C, Tree Walk III

Cómo atacarlo

Dos arreglos `izq` y `der`, indexados por ID, guardan el árbol, y -1 hace el papel de `NULL`. Los tres recorridos de la sesión se reescriben sobre índices: el caso base es `u == -1` y el recursivo baja a `izq[u]` y a `der[u]`. La raíz es el único ID que no aparece como hijo de nadie; se marca al leer. $\Theta(n)$ por recorrido y $O(h)$ de pila.

Las trampas

- Cada ID va precedido de un espacio: la línea empieza con espacio y no termina con uno.
- El enunciado no dice que la raíz sea 0 ni que las líneas lleguen en orden de ID. Hay que buscar la raíz.

Para el juez: Aizu ALDS1_7_B, Binary Tree I

Aizu ALDS1_7_B, Binary Tree (Aizu Online Judge)
Enunciado:
`https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_7_B`
Envío: `https://vjudge.net/problem/Aizu-ALDS1_7_B`

Qué pide

La misma entrada de ALDS1_7_C. Para cada nodo, en orden de ID, una línea con su padre, su hermano, su número de hijos, su profundidad, su altura y su tipo: `root`, `internal node` o `leaf`. Un dato que falta se imprime como -1.

Para el juez: Aizu ALDS1_7_B, Binary Tree II

¿Entendimos el problema?

Con el árbol del ejemplo, la línea del nodo 4 es

```
node 4: parent = 0, sibling = 1, degree = 2, depth = 1, height = 2,
internal node
```

Los nodos 1 y 4 tienen profundidad 1. ¿Por qué la altura de 4 es 2 y la de 1 es 1?

Bajo 4 está el camino 4, 5, 6, de dos aristas; bajo 1 solo hay hojas.

Para el juez: Aizu ALDS1_7_B, Binary Tree III

Cómo atacarlo

Al leer se llenan `izq`, `der` y `padre`. La profundidad va de arriba abajo: una función recibe el nodo y su profundidad d, la anota y llama a los hijos con $d + 1$, como `contarNivel`. La altura va de abajo arriba, con la `altura` de la sesión: -1 en el vacío. El hermano es el otro hijo del padre. Todo en $\Theta(n)$.

Las trampas

- La raíz se imprime como `root` aunque sea hoja, con `parent = -1` y `sibling = -1`.
- El hermano es -1 cuando el padre tiene un solo hijo.
- La altura cuenta aristas: una hoja tiene altura 0, como en la sesión.

Para el juez: Aizu ALDS1_7_D, Reconstruction of a Tree I

Aizu ALDS1_7_D, Reconstruction of a Tree (Aizu Online Judge)
Enunciado:
`https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_7_D`
Envío: `https://vjudge.net/problem/Aizu-ALDS1_7_D`

Qué pide

n nodos, $1 \leq n \leq 40$, con ID distintos de 1 a n. Llegan el preorden en una línea y el inorden en la siguiente; se imprime el posorden (Thareja, §9.4.5).

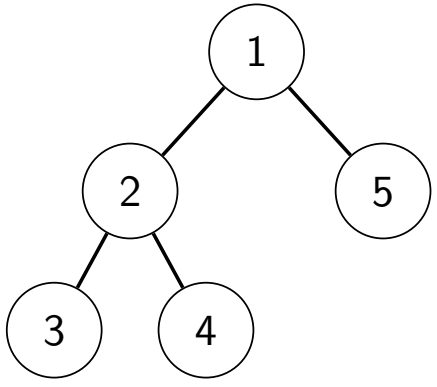
¿Entendimos el problema?

Preorden `1 2 3 4 5`, inorden `3 2 4 1 5`, posorden `3 4 2 5 1`. ¿Cómo se sabe que 5 está a la derecha de la raíz?

El preorden empieza por la raíz, 1. En el inorden, lo que está antes de 1 (`3 2 4`) es el subárbol izquierdo y lo que está después (`5`), el derecho.

Para el juez: Aizu ALDS1_7_D, Reconstruction of a Tree II

Rango del inorden	Raíz	Posición	Izquierdo	Derecho
$[0, 4]$	1	3	$[0, 2]$	$[4, 4]$
$[0, 2]$	2	1	$[0, 0]$	$[2, 2]$
$[0, 0]$	3	0	vacío	vacío
$[2, 2]$	4	2	vacío	vacío
$[4, 4]$	5	4	vacío	vacío



La raíz de cada rango es el siguiente número del preorden, que se consume en el orden de la tabla: 1, 2, 3, 4, 5.

Para el juez: Aizu ALDS1_7_D, Reconstruction of a Tree III

Cómo atacarlo

`reconstruir(ini, fin)` trabaja sobre el rango del inorden y un índice que avanza por el preorden, compartido por todas las llamadas. Caso base: `ini > fin`, el rango vacío. Caso recursivo: la raíz es el siguiente del preorden; se halla su posición m en el inorden, se resuelve $[ini, m-1]$, después $[m+1, fin]$, y al volver se imprime la raíz. El posorden sale sin crear nodos y sin copiar subarreglos: solo viajan índices. Buscar m recorriendo el rango da $O(n^2)$, de sobra con $n \leq 40$; un arreglo con la posición de cada ID lo baja a $\Theta(n)$.

Para el juez: Aizu ALDS1_7_D, Reconstruction of a Tree IV

Las trampas

- El izquierdo va antes que el derecho porque el preorden lista todo el subárbol izquierdo antes del derecho.
- Un solo espacio entre ID, sin espacio inicial: al revés de ALDS1_7_C.
- La raíz no siempre es 1. En el segundo ejemplo los dos recorridos son `1 2 3 4`: un árbol degenerado hacia la derecha, con posorden `4 3 2 1`.

Para el juez: Aizu ALDS1_7_D, Reconstruction of a Tree V

El mismo problema, con letras: UVa 536, Tree Recovery

Enunciado: `https://onlinejudge.org/external/5/536.pdf`
Envío: `https://vjudge.net/problem/UVA-536`

Cada línea trae el preorden y el inorden como dos palabras de letras mayúsculas distintas, y se imprime el posorden sin espacios. No se da el número de casos: se lee hasta el fin de la entrada. La posición de cada letra en el inorden cabe en un arreglo de 26.

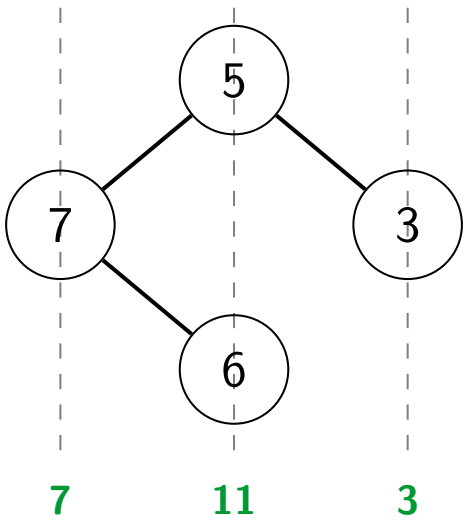
Para el juez: UVa 699, The Falling Leaves I

UVa 699, The Falling Leaves (UVa Online Judge)
Enunciado: `https://onlinejudge.org/external/6/699.pdf`
Envío: `https://vjudge.net/problem/UVA-699`

Qué pide

Cada hijo está una columna a la izquierda o a la derecha de su padre, y cada nodo deja caer tantas hojas como su dato. Se imprimen las sumas por columna, de izquierda a derecha. Cada árbol llega en preorden con -1 para el vacío; un -1 solo termina la entrada.

Para el juez: UVa 699, The Falling Leaves II



¿Entendimos el problema?

`5 7 -1 6 -1 -1 3 -1 -1` es este árbol, y su salida es `Case 1:` y en la línea siguiente `7 11 3`. ¿Por qué el 6 suma en la columna del 5?

El 6 es hijo derecho del 7, que está una columna a la izquierda del 5: una a la izquierda y una a la derecha lo devuelven a la columna 0, y $5 + 6 = 11$.

Para el juez: UVa 699, The Falling Leaves III

Cómo atacarlo

La lectura misma es el recorrido. `leer(col)` lee un número: si es -1, es un vacío y no hace nada (caso base); si no, lo suma en la columna `col` y llama `leer(col - 1)` para el subárbol izquierdo y `leer(col + 1)` para el derecho. Como la entrada viene en preorden, las llamadas piden los números en el orden en que llegan. No se crea ningún nodo. $\Theta(n)$ por árbol y $O(h)$ de pila.

Para el juez: UVa 699, The Falling Leaves IV

Las trampas

- Las columnas pueden ser negativas: el arreglo de sumas se indexa con `col` más un centro, y se recuerdan la columna mínima y la máxima usadas.
- Un árbol puede ocupar varias líneas; leer número por número lo resuelve sin pensar en líneas.
- Después de cada caso va una línea en blanco, también después del último.

Contenido I

1 Repaso: la tabla hash y el `map`

2 Datos que se contienen unos a otros

3 El TAD árbol

4 El árbol binario

5 La recursión, desde el árbol

6 Recorridos: preorden, inorden y posorden

7 Recorridos con pila explícita

8 Liberar el árbol

9 Ejercicios en parejas

10 Para el juez

11 Lo que cuesta

¿Cuántas llamadas hace `tamano`? I

Pregunta

`tamano(raiz)` sobre el árbol de ejemplo, de 8 nodos: ¿cuántas veces se llama `tamano` en total?

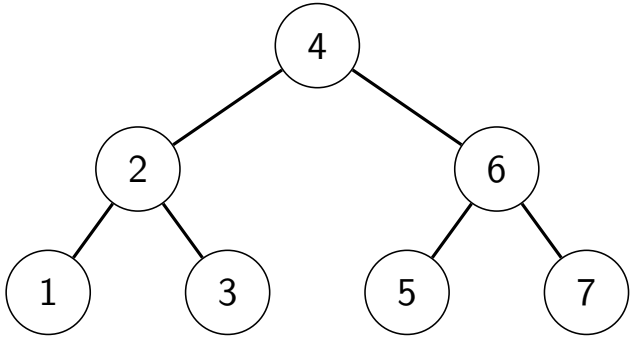
¿Cuántas llamadas hace `tamano`? II

Respuesta

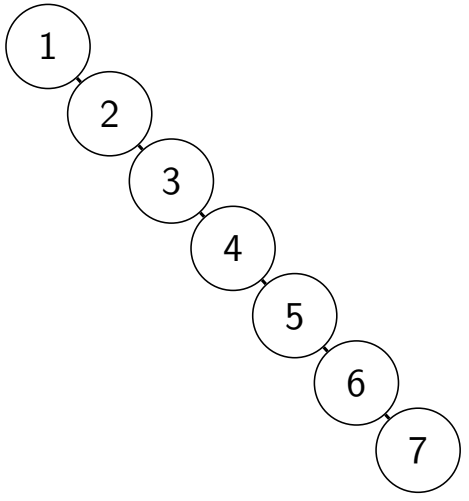
17: una por cada uno de los 8 nodos y una por cada uno de los 9 vacíos que se dibujaron con cuadros grises.

Con n nodos siempre hay $n+1$ vacíos. Los n nodos tienen $2n$ punteros; cada nodo distinto de la raíz recibe exactamente uno, el de su padre, así que $n-1$ punteros son aristas y los $2n-(n-1)=n+1$ restantes valen `NULL`. Son $2n+1$ llamadas, cada una con trabajo $\Theta(1)$ fuera de sus llamadas: $\Theta(n)$. La misma cuenta vale para `altura`, los tres recorridos, `liberar` y los ejercicios.

¿Cuánto vale h? I



lleno: $h = 2 = \lfloor \log_2 7 \rfloor$



degenerado: $h = 6 = n - 1$

El nivel k tiene a lo sumo 2^k nodos, así que con altura h caben a lo sumo $1 + 2 + \cdots + 2^h = 2^{h+1} - 1$. Con n nodos, $h \geq \lfloor \log_2 n \rfloor$; y como cada nivel tiene al menos un nodo, $h \leq n - 1$. El $O(h)$ de la pila va de $O(\log n)$ a $\Theta(n)$ según la forma, y el segundo extremo es el que desbordó la pila de llamadas.

Lo que cobra el árbol binario I

Operación	Tiempo	Espacio aparte	Por qué
`crearNodo`	$\Theta(1)$	$\Theta(1)$	un `new` y tres asignaciones
`tamano`, `altura`	$\Theta(n)$	$O(h)$	una llamada por nodo y por vacío; la pila guarda un camino desde la raíz
los tres recorridos	$\Theta(n)$	$O(h)$	lo mismo, con un `printf` por nodo
`liberar`	$\Theta(n)$	$O(h)$	un `delete` por nodo, en posorden
`evaluar`	$\Theta(n)$	$O(h)$	un posorden con una operación por nodo interno
`preordenPila`	$\Theta(n)$	$O(h)$	cada nodo entra y sale una vez; a lo sumo un derecho pendiente por nivel
`inordenPila`	$\Theta(n)$	$O(h)$	$2n$ vueltas; la pila es el camino de nodos que esperan
`posordenPila`	$\Theta(n)$	$\Theta(n)$	p2 guarda los n nodos antes de imprimir
`alturaPila`	$\Theta(n)$	$O(h)$	la pila es el camino de la raíz al nodo actual
ejercicios en parejas	$O(n)$	$O(h)$	a lo sumo una llamada por nodo y por vacío

Lo que cobra el árbol binario II

Espacio

La estructura ocupa $\Theta(n)$: un nodo por dato, con el dato y dos punteros. Las versiones recursivas piden aparte la pila de llamadas y las de pila explícita, nodos de `NodoPila`; en los dos casos, $O(h)$, salvo `posordenPila`.

Resumen I

- Un árbol con raíz se describe con padre, hijos, hermanos, hojas, grado, profundidad (desde arriba) y altura (desde abajo), contadas en aristas. Con hijo izquierdo y hermano derecho, cualquier árbol se guarda con dos punteros por nodo.
- Un árbol binario es vacío, o una raíz con un subárbol izquierdo y uno derecho. Una función sobre él tiene los mismos dos casos: el vacío responde sin llamar, y un nodo combina lo que devuelven sus subárboles.
- Cada llamada ocupa un marco en la pila de llamadas, y la pila guarda un camino desde la raíz: $O(h)$. Con un árbol degenerado de 175 000 nodos eso desborda los 8 MB de la pila.

Resumen II

- Preorden, inorden y posorden solo difieren en cuándo se imprime la raíz; sobre un árbol de expresión dan las notaciones prefija, infija y postfija. Los tres se escriben también con una pila de nodos, con la misma salida.
- Todos cuestan $\Theta(n)$ en tiempo: $2n + 1$ llamadas, una por nodo y una por vacío.

Resumen III

Lo que sigue

El árbol binario de búsqueda. El inorden del árbol de ejemplo salió `1 3 4 6 7 8 10 14`, ordenado, y el del árbol de los ejercicios, `2 9 7 5 4 3 6`, no. ¿Qué cumple cada nodo del primero, respecto de los datos de sus dos subárboles, que el segundo no cumple? ¿Cómo se buscaría el 7 en el primero sin recorrerlo entero?

Referencias I



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Apéndice B.5, árboles: §B.5.2 árboles con raíz y ordenados, §B.5.3 árboles binarios y posicionales; §10.3, representar árboles con raíz; §12.1, recorridos inorden, preorden y posorden.



R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 9, *Trees*: §9.1.1 terminología, §9.2.3 árboles binarios, §9.2.5 árboles de expresión, §9.4 recorridos, §9.4.5 reconstruir un árbol desde sus recorridos. Capítulo 10, *Efficient Binary Trees*: §10.2.4 a §10.2.6, altura, número de nodos y espejo.



Aizu Online Judge. ALDS1_7_C, *Tree Walk*: `https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_7_C`.



Aizu Online Judge. ALDS1_7_B, *Binary Tree*: `https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_7_B`.



Aizu Online Judge. ALDS1_7_D, *Reconstruction of a Tree*: `https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_7_D`.

Referencias II


UVa Online Judge. Problema 699, *The Falling Leaves*: `https://onlinejudge.org/external/6/699.pdf`.


UVa Online Judge. Problema 536, *Tree Recovery*: `https://onlinejudge.org/external/5/536.pdf`.